MCS-068
Predictive Data Analysis

Indira Gandhi National Open University
School of Computer and Information Sciences (SOCIS)

MCS-068PREDICTIVE DATA ANALYSIS

BLOCK 1: DATA ANALYSIS – AN INTRODUCTION

BLOCK 2: PREDICTIVE DATA ANALYSIS – I

BLOCK 3: PREDICTIVE DATA ANALYSIS – II

BLOCK 4: R - PROGRAMMING FOR DATA ANALYSIS

BLOCK 2: PREDICTIVE DATA ANALYSIS - I

UNIT 5: SIMILARITY MEASURES

UNIT 6: OUTLIER DETECTION

UNIT 7: PREDICTIVE ANALYTICS MODELS

,2026

ISBN – xx-xxxx-xxx-x

Printed and published on behalf of the Indira Gandhi National Open University, New Delhi by MPDD, IGNOU.

BLOCK INTRODUCTION (BLOCK 2)

Block-2, titled *Predictive Data Analysis-I*, builds upon the foundational concepts introduced in Block-1 and focuses on essential techniques required for predictive modeling. This block consists of three units (Unit 5 to Unit 7) and acts as a bridge between basic data analysis and advanced predictive analytics. It introduces important computational and statistical methods used for handling complex datasets and preparing data for model development.

The block begins with **Unit-5: Similarity Measures**, which explains techniques used to measure relationships and similarities between data points, sets, and documents. It covers concepts such as Jaccard similarity, cosine similarity, Euclidean distance, Hamming distance, and edit distance, along with advanced methods like shingling, min-hashing, and locality-sensitive hashing. These techniques are widely used in recommendation systems, clustering, and document comparison.

Unit-6: Outlier Detection focuses on identifying unusual or abnormal data points that differ significantly from normal patterns. The unit includes statistical methods such as standard deviation, Z-score, box plots, and percentiles, along with advanced approaches like DBSCAN and Local Outlier Factor (LOF). Outlier detection is important for improving data quality, increasing model accuracy, and ensuring reliable analytical outcomes.

Finally, **Unit-7: Predictive Analytics Models** introduces different categories of predictive models, including regression models, time series forecasting, supervised learning, and unsupervised learning techniques. This unit provides learners with a conceptual understanding of predictive modeling approaches and prepares them for advanced topics in later blocks.

The importance of this block is significant in both industry and academia. In industry, similarity measures are widely used in recommendation systems and customer analytics, while outlier detection supports fraud detection, risk analysis, and quality management. Predictive models help organizations forecast trends and make data-driven decisions. In academia, this block provides the theoretical and methodological foundation for advanced studies in data science, machine learning, and artificial intelligence.

Overall, Block-2 equips learners with essential analytical methods and predictive modeling concepts required for solving real-world data problems and progressing toward advanced predictive analytics techniques.

UNIT 5 MINING BIG DATA

Structure **Page No.**

5.0 INTRODUCTION

In the previous Block, you have gone through the concepts of Big data and Big data handling frameworks. These concepts include the distributed file system, MapReduce and other similar architectures. This Block focuses on some of the techniques that can be used to find useful information from Big data.

This Unit focuses on the issue of finding similar item sets in Big data. The Unit also defines the measures for finding the distances between two data objects. Some of these techniques include Jaccard distance, Hamming's distance etc. In addition, the Unit also discusses finding similarities among the documents using shingles. Finally, this unit introduces you to some of the other techniques that can be used for analysing Big data.

5.1 EXPECTED LEARNING OUTCOMES

After going through this Unit, you will be able to:

- Explain different techniques for finding similar items
- Explain the process of the collaborative filtering process
- Use shingling to find similar documents
- Explain various techniques to measure the distance between two objects
- Define supervised and unsupervised learning

5.2 FINDING SIMILAR ITEMS

Finding similar items for a small set of documents may be a simple problem, but how do you find a set of similar items when the number of items is extremely large? This section defines the basic issues of finding similar items and their applications.

Problem Definition:
Given an extremely large collection of item sets, which may have millions or billions of sets, how to find the set of similar item sets without comparing all the possible combinations of item sets, using the notion that similar item sets may have many common sub-sets.

Purpose of Finding Similar Items:

1. You may like to classify web pages that are using similar words. This information can be used to classify web pages.
2. You may like to find the purchases and feedback of customers on similar items to classify them into similar groups, leading to making recommendations for purchases for these customers.
3. Another interesting use of similar item set is in entity resolution, where you need to find out if it is the same person across different applications like e-Commerce web sites, social media, searches etc.
4. Two or more web pages amongst millions of web pages may be identical. These web pages may be plagiarized or a mirror of a simple website.

Why Finding Similar Items is a problem?

One of the simplest ways to find similar items would be to compare two documents or web pages and determine if they are identical or not by comparing sequences of characters/words used in those documents/web pages. However, considering that you need to find duplicates in 10 documents/web pages, you need to compare ${}^{10}C_2 = 45$ pairs. In general, for n documents/webpages, you may need to compare $n \times (n-1)/2$ pairs of documents/web pages. For about 10^6 documents/web pages, you need to make about 5×10^{11} comparisons. Thus, the question is how to find these identical documents/web pages amongst billions of documents/web pages without checking all the combinations.

Next, we first discuss a technique to find the similarity of sets, followed by techniques that can be used to find similar item sets efficiently.

5.3 FINDING SIMILAR SETS

In this section, we discuss one of the measures of finding similarity among the sets and then discuss how this similarity measure can be used for finding the textual similarity of documents; and collaborative filtering, which can be used for finding a similar group of customers.

5.3.1 Jaccard Similarity of Sets

Jaccard similarity is defined in the context of sets. Consider two sets – set A and set B, then the Jaccard similarity $JS_{A,B}$ is defined as a ratio of the cardinality of the set $A \cap B$ and cardinality of the set $A \cup B$. The value of $JS_{A,B}$ can vary from 0 to 1. The following equation represents the Jaccard similarity:

$$JS_{A,B} = \frac{|A \cap B|}{|A \cup B|}$$

For example, consider the sets A={a, b, c, d, e} and set B={c, d, e, f, g}, then Jaccard similarity of these two sets would be:

$$JS_{A,B} = \frac{|\{c,d,e\}|}{|a,b,c,d,e,f,g|} = \frac{3}{7}$$

5.3.2 Documents Similarity

Finding document similarity is an interesting domain of use of Jaccard similarity. It can be used to identify a collection of almost similar documents, news items, web pages or reports. This is also referred to as the character-level similarity of documents. Another kind of document similarity also looks at documents having similar meanings. This kind of similarity requires you to identify similar words and sometimes the meaning of the words. This is also a very useful similarity, but it can be solved using different types of techniques. How can you identify, if two documents are identical? This can be done simply by character-by-character matching. However, this algorithm may be of little use as many documents may be mostly similar though not exactly. The following cases specify this situation:

Plagiarism Checking: The plagiarized text may differ in small segments, as some portions of the document may have been changed or even the ordering of some of the sentences may be changed.

Mirror Websites: Even the mirror websites also make changes, as per the local advertisements and requirements.

The versions of Course Material: Even the versions of older course material and newer course materials, assignments etc. available on different websites may be slightly different.

Similar News Reports: The news reports about a news item may consist of similar text, which may be appended with supplementary information by each newspaper.

5.3.3 Collaborative Filtering and Set Similarity

In the past decade, e-commerce has gained acceptance by customers. Many of you, who have purchased and have liked various items on these e-commerce websites, get online recommendations for the purchase of certain items. You may have observed that many times those recommendations are very close to purchases that you would like to do in the near future. This is achieved by the process of collaborative filtering, which tries to group you based on your purchases and likes with other users making similar purchases and likes and then recommending you the products that another person in the group has purchased and liked.

Similarity of the sets is used to address this problem. Two customers can be in a similar customer group if they purchase and like similar products. This similarity is determined by the Jaccard set similarity. However, please note that the number of customers, as well as the products that can be purchased on an e-commerce website, are very large. For example, two separate customers may be interested in purchasing books. They may purchase the same books available on Data Science and may rate them similarly. Please note they may also purchase many other books, which may be bought by only one of them. How are the problems of collaborative filtering and document similarity different? The prime difference is in the value of Jaccard similarity. For document similarity, you are looking at $JS_{A,B}$ in the range of 50% or above, whereas for the case of collaborative filtering, you may be interested if the value of $JS_{A,B}$ is as low as 10-25%.

In general, collaborative filtering uses binary data, such as like/dislike or seen/not seen or purchased/not purchased etc. This data can easily be translated into a set such as a set of likes of items; a set of purchased items; etc. Thus, finding Jaccard's similarity is straightforward. However, in many situations, a 5-, 7- or 11-point Likert scale may be used to rate the articles. An interesting way to deal with such data may be to create a bag instead of a set having as many instances of an item as the rating. For example, consider that on a 5-point scale, a customer gave a rating of 2 to product A and a rating of 3 for product B and another customer gave a rating of 1 to product A and a rating of 4 for product B, then the related data would be represented as:

BagCust1 = {a, a, b, b, b}
BagCust2 = {a, b, b, b, b}

$$JS_{A,B} = \frac{|\{a,b,b,b\}|}{|a,a,b,b,b,b|} = \frac{4}{6} = \frac{2}{3}$$

Thus, the set similarity is an important consideration for finding the similarity between two sets. In the next section, we discuss the method to convert a document into subsets of very small strings, which can be used to compute the similarity of two documents.

Check Your Progress 1:

Question 1: What is the major problem in finding similar items?
Question 2: Define Jaccard Similarity with the help of an example.
Question 3: What is collaborative filtering? How can it be addressed using set similarity?

5.4 FINDING SIMILAR DOCUMENTS

In the last section, we discussed the Jaccard similarity in the context of sets. We also explained the issues of document similarity. The document similarity can be checked using the following process:

Step 1: Create Sets of items from the document (Shingles are used)
Step 2: Perform Minhashing and create documents Signatures that can be tested for similarity checking. This reduces the number of shingles to be tested for checking the similarity of two documents.
Step 3: Perform Locality Sense Hashing produces pairs of documents that should be checked for similarity rather than all the possible pairs

In this section, we discuss these three important concepts of document similarity analysis.

5.4.1 Shingles

In order to define the term shingle, let us first try to answer the question: How to represent a document as a set of items so that you can find lexicographically similar documents?

One way would be to identify words in the document. However, the identification of words itself is a time-consuming problem and would be more useful if you are trying to find the semantics of sentences. One of the simplest and efficient ways would be to divide the document into smaller substrings of characters, say of size 3 to 7. The advantage of this division is that for almost common sentences many of these substrings would match despite small changes in those sentences or changes in the ordering of sentences.

A k-shingle is defined as any substring of the document of size k. A document has many shingles, which may occur at least once. For example, consider a document that consists of the following string:

"bit-by-bit"

Assuming the value of k=3 and even using a blank character as part of substrings. The following are the possible substrings of this document of size 3:

"bit", "it-", "t-b", "-by", "by-", "y-b", "-bi", "bit"

Please note that out of these substrings "bit" is occurring twice. Therefore, the 3-shingles for this document would be:

{"bit", "it-", "t-b", "-by", "by-", "y-b", "-bi"}

One of the issues while making shingles is how to deal with white spaces. A possible solution, which is used commonly, would be to replace all the continuous white space characters with a single blank space.

So, how do you use shingles to find similar documents?

You may convert the documents into shingles and check if the two documents share the same shingles.

An interesting issue here would be to determine the size of the shingle. If the size of the shingle is small then most of the documents would have those shingles, even if they are not identical. For example, if you keep a single size of k=1, then almost all the characters would be shingles and these shingles would be common in most of the documents, as most of them will have almost all the alphabets. On the other hand, bigger shingles may not be able to distinguish similar items. The ideal size of a shingle is k=5 to 9 for small to large documents.

How many different shingles are possible for a character set? Assume that a typical language has n characters and the size of shingles is k, then the possible number of shingles is n^k. Thus, the number of possible shingles in a document may be very large.

Consider that you are using 9-shingles for finding similar research articles amongst very large size articles. The possible character set would require 26 alphabets of the English language and one character for the space character. Therefore, the maximum possible set of shingles would have $(26+1)^9 = 27^9$ 9-shingles with each shingle being 9 bytes long (assuming 1 alphabet = 1 byte). This is a very large set of data.

One of the ways of reducing the data would be to use hash functions instead of shingles. For example, assume that a hash function maps a 9-byte substring to an integral hash bucket number. Assuming that the size of the integer is 4 bytes, then the hash function maps these 27^9 possible 9-shingles to $2^{4*8}-1=2^{32}-1$ possible buckets. You may now use the bucket number as the shingle itself, thus reducing the size of the shingle from 9 bytes to 4 bytes.

For certain applications, such as finding similar news articles, shingles are created based on the words. These shingles are found to be more effective for finding the similarity of news articles.

5.4.2 Minhashing

As you can observe, the set of shingles of a document, in general, is large. Even if we reduce this set using minhashing of shingles to a smaller size, as discussed in the previous section, the size is substantial. In fact, in general, the total size of the set of shingles in Bytes is larger than the size of the document. Thus, it will not be possible to accommodate the shingles in the memory of a computer, so as to find the similarity of the documents. Is it possible to reduce

this stored size of shingles, yet not compromise too much on the estimates of Jaccard similarity?

The method used to do so is called minhashing. The idea here is to convert the set of shingles to a set of signatures of documents using a large number of permutations of rows of the matrix. In order to explain this process, let us use a visual representation of shingles using a matrix. Please note the word visual representation, as it is being used only for explanation. The actual representation would be closer to the representation of a sparse matrix. Consider a universal document that has 5 elements or shingles. The four document sets include the following shingles

Document set 1 = {2, 3, 5}
Document set 2 = {3, 4}
Document set 3 = {1, 4}
Document set 4 = {1, 2, 4}

These documents can be represented using the following matrix of Shingles and the documents:

Shingle/Documents	Set 1	Set 2	Set 3	Set 4
1	0	0	1	1
2	1	0	0	1
3	1	1	0	0
4	0	1	1	1
5	1	0	0	0

Figure 1: Matrix Representation of Documents and associated shingles

You can compute the Jaccard similarity of these documents or sets as follows:

$$JS_{SetA,SetB} = \frac{The\ number\ of\ rows\ having\ 1's\ in\ columns\ of\ BOTH\ the\ sets}{The\ number\ of\ rows\ having\ 1'\ sin\ sin\ at\ least\ one\ of\ the\ two\ column\ of\ the\ sets}$$

Please note that in the equation given above the columns of the sets having 0s in both columns are not counted, as they show that that shingle is NOT present in both columns.

Thus, for computing the similarity of set 1 and set 2 (see Figure 1), you may observe that row 1 has values 0 in the columns of set 1 and set 2 both, therefore, would not be counted. You may also observe that only row 3 has a value of 1 in columns of set 1 and set 2 both. In all the other rows only one of the two columns is 1. This indicates that only shingle 3 is present in both documents. Thus, the similarity would be:

$$JS_{Set1,Set2} = \frac{1}{4}; \qquad JS_{Set1,Set3} = \frac{0}{5}; \qquad JS_{Set1,Set4} = \frac{1}{5}$$

$$JS_{Set2,Set3} = \frac{1}{3}; \qquad JS_{Set2,Set4} = \frac{1}{4}$$

$$JS_{Set3,Se} = \frac{2}{3}$$

You can verify the Jaccard similarity from the set definitions also. Thus, using the matrix representation, you may be able to compute the similarity of two documents or sets.

After going through the representation and its Jaccard similarity, next, let us define the term minhashing. You can create a large number of orderings of the rows of the shingle/document matrix given in Figure 1, to generate a new sequence of rows or shingles (for our example). The first non-zero shingle of each set is called the minhashed value of that set. For example, consider the following three ordering of the matrix:

ORDER	Shingle/Documents	Set 1	Set 2	Set 3	Set 4
1st	2	1	0	0	1
2nd	4	0	1	1	1
3rd	3	1	1	0	0
4th	5	1	0	0	0
5th	1	0	0	1	1
MH_1(Set n)	-	1st	2nd	2nd	1st

ORDER	Shingle/Documents	Set 1	Set 2	Set 3	Set 4
1st	5	1	0	0	0
2nd	1	0	0	1	1
3rd	3	1	1	0	0
4th	2	1	0	0	1
5th	4	0	1	1	1
MH_2(Set n)	-	1st	3rd	2nd	2nd

ORDER	Shingle/Documents	Set 1	Set 2	Set 3	Set 4
1st	1	0	0	1	1
2nd	2	1	0	0	1
3rd	4	0	1	1	1
4th	3	1	1	0	0
5th	5	1	0	0	0
MH_1(Set n)	-	2nd	3rd	1st	1st

Figure 2: Computation of Minhash Signatures

You may collect these minhashed values, called signatures, into a signature matrix. Thus, a typical signature matrix of the minhasing as above is given in Figure 3.

Set 1	Set 2	Set 3	Set 4
1st	2nd	2nd	1st
1st	3rd	2nd	2nd
2nd	3rd	1st	1st

Figure 3: Minhash Signature Matrix

Likewise, you can create a large number of minhashed values. But how can you compute the Jaccard similarity using these minhashed values?

In order to answer this question, you may think about the probability of finding the same minhash signatures in two columns. This would happen when both the columns have value 1 in the row, which has been selected as the first non-zero row for both columns. Further, the minhash signatures will be different if you have selected a row, which has only one column having a value 1 and the other having a value 0. Thus, going by this logic, if you select a very large number of orderings of the row, the probability of a similar minhash value of two columns would be the same as the Jaccard similarity. Thus, you can reduce the problem of finding the similarity of documents to finding the similarity of minhash signatures. Thus, reducing the complexity of the problem.

For example, you can compute the approximate similarity using Figure 3, as follows:

$$AJS_{Set\ ,Set2} = \frac{0}{3}; \qquad AJS_{Set1,Set3} = \frac{0}{3}; \qquad AJS_{Set\ ,Se} = \frac{1}{3}$$

$$AJS_{Set2,Set3} = \frac{1}{3}; \qquad AJS_{Set2,Set4} = \frac{0}{3}$$
$$AJS_{Set3,Set4} = \frac{2}{3}$$

Thus, you may observe that the Jaccard Similarity can be computed approximately using the signature matrix, which can be used to determine the similarity between two documents.

Now, consider the case when you are computing the signatures for 1 million rows of data. The ordering of these itself is time-consuming and representing an ordering will require large storage space. Thus, for real data, the presented algorithm may not be practical. Therefore, you would like to simulate the ordering using simple hash functions rather than actual ordering. You can decide the number of buckets, say 100, and hash the matrix (see Figure 1) into these buckets. You must select different hash functions such that a column of a row (if it has a value 1) is mapped to a different bucket by a particular hash function. This will ensure that hash functions create different ordering in the buckets. Please remember that hashing may result in collisions, but by using many hash functions, the effect of collision can be minimized. You must select the smallest bucket number to which a column is mapped as its minhash signature value. The following example explains the process with the help of two hash functions. Consider the documents and shingles given in Figure 1, and assume two hash functions given below:

$$h_1(x) = (x + 1)\ mod\ 5$$
$$h_2(x) = (2x + 3)\ mod\ 5$$

We assume 5 buckets, numbered 0 to 4. The following signatures can be created by using these hashed functions:

Initial Values:

	Set 1	Set 2	Set 3	Set 4
h_1	∞	∞	∞	∞
h_2	∞	∞	∞	∞

Figure 4: Initial hashed values

Now, apply the hash function on the first row of Figure 1. Since Set 1 and Set 2 have values 0, therefore, the value in the above table will not change. However, Set 3 and Set 4 would be mapped as follows:

For Set 3 and Set 4:

$$h_1(1) = (1 + 1)\ mod\ 5 = 2$$
$$h_2(1) = (2 * 1 + 3)\ mod\ 5 = 0$$

Figure 4 will change now to:

	Set 1	Set 2	Set 3	Set 4
h_1	∞	∞	2	2
h_2	∞	∞	0	0

Figure 5: Hashed signature after hashing the first row

Applying the hash function on the second row of Figure 1:

For Set 1 and Set 4:

$$h_1(2) = (2 + 1)\ mod\ 5 = 3$$
$$h_2(2) = (2 * 2 + 3)\ mod\ 5 = 2$$

Since Set 4 already has lower values than these so no change will take place in set 4. Figure 5 will be modified as:

	Set 1	Set 2	Set 3	Set 4
h_1	3	∞	2	2
h_2	2	∞	0	0

Figure 6: Hashed signature after hashing the second row

Applying the hash function on the third row of Figure 1:
For Set 1 and Set 2:

$$h_1(3) = (3 + 1)\ mod\ 5 = 4$$
$$h_2(3) = (2 * 3 + 3)\ mod\ 5 = 4$$

Since Set 1 already has lower values than these so no change will take place in set 1. Figure 6 will be modified as:

	Set 1	Set 2	Set 3	Set 4
h_1	3	4	2	2
h_2	2	4	0	0

Figure 7: Hashed signature after hashing the third row

Applying the hash function on the fourth row of Figure 1:
For Set 2, Set 3 and Set 4:

$$h_1(4) = (4 + 1)\ mod\ 5 = 0$$
$$h_2(4) = (2 * 4 + 3)\ mod\ 5 = 1$$

This will result in changes in the h_1 values of Set 2, Set 3 and Set 4 and h_2 value of Set 2 only, as h_2 values of Set 3 and Set 4 are already lower. Figure 7 will be modified as:

	Set 1	Set 2	Set 3	Set 4
h_1	3	0	0	0
h_2	2	1	0	0

Figure 8: Hashed signature after hashing the fourth row

Applying the hash function on the fifth row of Figure 1:
For Set 1 only:

$$h_1(5) = (5 + 1)\ mod\ 5 = 1$$
$$h_2(5) = (2 * 5 + 3)\ mod\ 5 = 3$$

This will result in changes in the h_1 values of Set 1 only. Figure 8 will be modified as:

	Set 1	Set 2	Set 3	Set 4
h_1	1	0	0	0
h_2	2	1	0	0

Figure 9: Hashed signature after hashing the fifth row

The signature so computed will give almost the same similarity measure as earlier. Thus, we are able to simplify the process of finding the similarity between two documents. However, still one of the problems remains, i.e., there are a very large number of documents between which the similarity is to be checked. The next section explains the process of minimizing the pairs that should be checked for similarities.

5.4.3 Locality Sensitive Hashing

The signature matrix as shown in Figure 9 can also be very large, as there may be many millions of documents or sets, which are to be checked for similarity. In addition, the number of minhash functions that can be used for these documents may be in the hundreds to thousands. Therefore, you would like to compare only those pairs of documents that have some chance of similarity. Other pairs of documents will be considered non-similar, though there may be small false negatives in this group. Locality-sensitive hashing is a technique to find possible pairs of documents that should be checked for similarity. It takes the signature matrix, as shown in Figure 9, as input and produces the list of possible pairs, which should be checked for similarity. The locality-sensitive hashing uses hash functions to do so.

The basic principle of the Locality-sensitive hashing technique is as follows:

Step 1: Divide the overall set of the signature matrix into horizontal portions of sets, let us call them horizontal bands. This will reduce the effective size of data that is to be processed at a time.

Step 2: Use a separate hash function to hash the column of a horizontal band into a hash bucket. It may be noted that if two sets are similar, then there is a very high probability that at least one of their horizontal band will be hashed by some hash function to the same bucket.

Step 3: Perform the similarity checking on the pairs of sets collected in each bucket. You may please note that the probability of hashing dissimilar sets to the same bucket is low.

Thus, finally reducing the pairs that are to be checked for similarity can be reduced using locality-sensitive hashing.

You may please note that this method is an approximate method, as there would be a certain number of false positives as well as false negatives. However, given the size of the data, a small probability of errors are acceptable in the results.

Check Your Progress 2:

Question 1: What is a shingle? What are its uses? List all the 5-shingle of a document, which contain the text "This is a test"

Question 2: Consider the following Matrix representation of 2 documents and their associated shingles. What is the Jaccard Similarity of the documents?

Shingle/Documents	Set 1	Set 2
1	0	0
2	1	1
3	1	1
4	1	1
5	1	0

Question 3: Consider any three orderings of the sets to compute the minhash signature of the documents.

Question 4: Compute the signature matrix from minhash signatures and compute the similarity using the signature matrix.

Question 5: What is Locality sensitive hashing?

5.5 Distance Measures

In the previous sections, we used the Jaccard similarity to find similarity measures. In this section, we define some of the important distance measures that are used in data science. This list is not exhaustive, you may find more such measures from further readings.

The term distance is defined with reference to space, which is defined as a set of points.

A distance measure, say *distance(x, y)*, is the distance between two points in space. Some of the basic characteristics of this distance are:

1. Distance is always positive. It can be zero, only if *x* and *y* are the same points.
2. The distance measured between two points is symmetrical i.e. *distance(x, y)* is the same as *distance(y, x,)*.
3. The distance between two points would be the distance of the shortest path between two points. This can be represented with the help of a third point, say t, by the following equation:

$$distance(x, y) \leq distance(x, t) + distance(t, y)$$

Let us discuss some of the basic distance measures in the following sub-sections.

5.5.1 Euclidean Distance

In Euclidean space, a point can be represented as an n-dimensional vector. For example, a point *x* can be represented as an n-dimensional vector as $x(x_1,x_2, x_3,...x_n)$. The distance of two n-dimensional points *x* and *y* in Euclidean n-dimensional space, where *x* is represented as $x(x_1,x_2, x_3,...x_n)$ and *y* is represented as $y(y_1,y_2, y_3,...y_n)$, can be computed using the following equation:

$$distance(x, y) = \left(\sum_{i=1}^{n} [\{|(y_i - x_i)|\}^m]^{\frac{1}{m}}\right)$$

In the case of m = 1, you get the formula:

$$manhattendistance(x, y) = \left(\sum_{i=1}^{n} |(y_i - x_i)|\right)$$

This is called the Manhattan distance.

In case of m =2, we get the formula:

$$distance(x, y) = \left(\sqrt{\sum_{i=1}^{n} (y_i - x_i)^2}\right)$$

You can verify that these measures satisfy all the properties of the distance measure.

5.5.2 Jaccard Distance

As discussed earlier, you compute the Jaccard similarity of two sets, namely set A and set B, using the following formula:

$$JS_{A,B} = \frac{|A \cap B|}{|A \cup B|}$$

The Jaccard distance between these two sets is computed as:

$$JaccardDistance(A,B) = 1 - JS_{A,B} = 1 - \frac{|A \cap B|}{|A \cup B|}$$

Does this distance measure fulfil all the criteria of a distance measure?
The value of Jaccard Similarity varies from 0 to 1, therefore, the value of Jaccard distance will be from 1 to 0. The value 0 of Jaccard distance means that the value of $A \cap B$ will be the same as $A \cup B$, which can occur only if set A and set B are identical. In addition, as set intersection and union both are symmetrical operations, therefore the Jaccard distance is also symmetrical. You can also test the third property using certain values for three sets.

5.5.3 Cosine Distance

The Cosine distance is computed for those spaces where points are represented as the direction of vector components. In such cases, the cosine distance is defined as the angle between the two points. You can use the dot product and magnitude of the vectors to compute the cosine distance between two vectors. For example, consider the two vectors A[1,0,1] and B[0,1,0], the cosine distance between the two vectors can be computed as:

The dot product of the two vectors A.B = 1×0+0×1+1×0=0

$$A.B = |A| \times |B| \cos\theta$$

$$0 = \sqrt{2} \times 1 \cos\theta$$

$$\cos\theta = 0 \text{ or } \theta = 90°$$

You may check if this measure also fulfils the criteria of the measures. It may be noted that the range of cosine distance is evaluated between 0 and 180 degrees only. You may go through further readings for more details.

5.5.4 Edit Distance

The edit distance may be used to determine the distance between two strings. It can be defined as follows:
Consider two strings, string A consisting of n characters $a_1, a_2, \ldots a_n$ and a string B of m characters $b_1, b_2, \ldots b_m$, then edit distance between the two strings is the minimum number operations that are required to make strings identical. These operations are either the addition of a single character or the deletion of a single character.
For example, the edit distance between the strings: A= "abcdef" and B= "acdfg", would be 3, as you can obtain B from A; by deleting b, deleting e and inserting g at the end in string A.

However, finding these operations may not be easy to code, therefore, edit distance can be computed using the following method:
Step 1: Consider two strings - string A consisting of *n* characters $a_1, a_2, \ldots a_n$ and a string B of *m* characters $b_1, b_2, \ldots b_m$. Find a sub-sequence, which is the longest and has the same character sequence in the two strings.

Use the deletion of character operation to do so. Assume the size of this sub-sequence is *ls*

Step 2: Compute the edit distance using the formula:

$$editdistance(A, B) = n + m - 2 \times ls$$

For example, in the strings A= "abcdef" and B= "acdfg", the longest common sub-sequence is "acdf", which is 4 characters long, therefore, the edit distance between the two strings is:

$$editdistance(A, B) = 6 + 5 - 2 \times 4 = 3$$

You can verify that edit distance is a proper distance measure.

5.5.5 Hamming Distance

Hamming distance is one of the most used distances in digital design and error detection in digital systems, e.g. while mapping of min-terms into Karnaugh's map or single error correcting code. The Hamming distance is defined as the difference between the components of two vectors, especially when these vectors are of type Boolean.

For example, consider the following two Boolean vectors:

Vector A	1	0	1	1	0	0	1	1
Vector B	1	1	0	1	0	0	0	0
Difference	N	Y	Y	N	N	N	Y	Y

The Hamming distance between the vectors A and B is 4.
You may verify that Hamming distance also satisfies the properties of a distance measure.

You may use any of these distance measures based on the type of data being used or the type of problem.

5.6 INTRODUCTION TO OTHER TECHNIQUES

Big data analytics are the processes that are used to analyse Big data, which include structured and unstructured data, to produce useful information for organisational decision-making. In this section, we introduce some of the techniques that are useful for extracting useful information from Big data. You may refer to the latest research articles from various journals for more details on such techniques.

Textual Analysis:

Textual data is produced by a large number of sources of Big data, e.g. email, blogs, websites, etc. Some of the important types of analytics that you may perform on the textual data are:

1. Structured Information Extraction from the unstructured textual data: The purpose here is to generate information from the data that can be stored for a longer duration and can be reprocessed easily if needed. For example, the Government may find the list of medicines that are being prescribed by doctors in various cities by analysing the prescriptions given by the doctors to different patients. Such analysis would essentially require recognition of

entities, such as doctor, patient, disease, medicine etc. and then relationships among these entities, such as among doctor, disease and medicine.

2. Meaningful summarization of single or multiple documents: The basic objective here is to identify and report the key aspects of a large group of documents, e.g. financial websites may be used to generate data that can be summarised to produce information for stock analysis of various companies. In general, the summarization techniques either extract some important portions of the original documents based on the frequency of words, location of words etc.; or semantic abstraction-based summaries, which use Artificial Intelligence to generate meaningful summaries.
3. Question-Answering: In the present time, these techniques are being used to create automated query-answering systems. Inputs to such systems are the questions asked in the natural languages. These inputs are processed to determine the type of question, keywords or semantics of the questions and possible focus of the question. Next, based on the type of the Question-Answering system, which can be either an Information Retrieval based or a knowledge-based system, either the related information is retrieved or information is generated. Finally, the best possible answer is sent to the person who has asked the question. Some examples of Question-answering systems are – Siri, Alexa, Google Assistant, Cortona, IBM Watson etc.
4. Sentiment Analysis: Such analysis is used to determine the opinion or perception of feedback about a product or service. Sentiment analysis at the document level determines if the given feedback is positive feedback or negative feedback. However, techniques have been developed to perform sentiment analysis at the sentence level or aspect level.

Audio-Video Analysis:

The purpose of audio or video analysis is to extract useful information from speech data or video data. Most of the calls to the customer call centres are recorded for performance enhancement. One of the common approaches to dealing with such data is to transcribe the speech data using automated speech recognition software. Such software converts the speech into text, which is checked in a dictionary to ascertain if such a word exists or not. Even phonetics can be used for converting speech to text.
Video analysis is performed on video or closed-circuit television (CCTV) data. The objective of such data analysis is to check for security breaches, which may be checked by changes in CCTV data if any. In addition, it can be used to perform indexing of videos, so as to find relevant information at a later time.

Social Media Data Analysis:

This is one of the largest chunks of present-day data. Such data can be part of social networks, micro-blogging, wiki-based content and many other related web applications. The challenge here is to obtain structured information from noisy, user-oriented, unstructured data available in a large number of diverse

and dispersed web pages to produce information like finding a connected group of people, community detection, social influence, link predictions etc. leading to applications like recommender systems.

Predictive Analysis:

The purpose of such analysis is to predict some of the patterns of the future based on the present and past data. In general, predictive analysis uses some statistical techniques like moving averages, regression and machine learning. However, in the case of Big data, as the size of the data is very large and it has low veracity, you may have to develop newer techniques to perform predictive analysis.

Supervised and Unsupervised Learning:

Some of the Big data problems can also be addressed using machine learning algorithms, which create models by learning from the enormous data. These models are then used to make future predictions. In general, these algorithms can be classified into two main categories – Supervised Learning algorithms and unsupervised learning algorithms.

Supervised Learning:
Supervised learning uses already available knowledge to generate models, one of the most common examples of supervised learning is spam detection in which the word patterns and anomalies of already classified emails (spam or not-spam) are used to develop a model, which is used to detect if a newly arrived email is spam or not. Supervised learning problems can further be categorised into classification problems and regression problems.

Unsupervised Learning:
Unsupervised learning generates its own set of classes and models, one of the most common examples of unsupervised learning is creating a new categorisation of customers based on their feedback on different types of products. This new categorisation may be used to market different types of products to these customers. Such types of problems are also called clustering problems.

You can obtain more details on supervised and unsupervised learning in the AI and Machine learning course (MCS224).

Check Your Progress 3:

1. What is the purpose of a distance measure? What are the characteristics of distance measures?
2. What is the Cosine distance measure? How is it different from Hamming's distance?
3. Explain the purpose of text analysis.

5.7 SUMMARY

This unit introduces you to basic techniques for finding similar items. The Unit first explains the methods of finding similarity between two sets. In this context, the concept of Jaccard similarity, document similarity and collaborative similarity are

discussed. This is followed by a detailed discussion on one of the important set similarity applications – document similarity. For finding document similarity of a very large number of documents, a document is first converted to a set of shingles, next the minhashing function is used to compute the signatures of document sets against these shingles. Finally, locality-sensitive hashing is used to find the sets that must be compared to find similar documents. The locality-sensitive hashing greatly reduces the number of documents that should be compared to find similar documents. The Unit then describes several common distance measures that may be used in different types of Big data analysis. Some of the distance measures defined are Euclidean distance, Jaccard distance, Cosine distance, Edit distance and Hamming distance. Further, the unit introduces you to some of the techniques that are used to perform analysis of Big data.

5.8 SOLUTIONS/ANSWERS

Check Your Progress 1:

1. For finding similar items, e.g. web pages, the first problem is the representation of web pages into the sets, which can be compared. The major problem, in finding similar items, is the size of the data. For example, if you find duplicate web pages, you may have to compare billions of pairs of web pages.
2. You can define Jaccard's similarity between two sets, viz. Set A and Set B as:

$$JS_{A,B} = \frac{|A \cap B|}{|A \cup B|}$$

Consider two sets A={a, b, c, d, e, f, g} and B={a, c, e, g, i}, then Jaccard similarity between the two sets is:

$$JS_{A,B} = \frac{|\{a,c,e,g\}|}{|\{a,b,c,d,e,f,g,i\}|} = \frac{4}{8} = \frac{1}{2}$$

3. Collaborative filtering is the process of grouping objects based on certain criteria and providing possible suggestions to those objects based on the activities of other objects in the group. The similarity is the basis on which these groups are constructed.

Check Your Progress 2:

1. Shingle is a small string of size 3-5 characters, which can be part of a document. Shingles are used to convert a document into sets, which can be checked for similarity. The document containing the text "This is a test document" has the following set of 5-shingles (_ is used to represent a space character):
   {This_, his_i, is_is, s_is_a, _is_a, is_a_, s_a_t, _a_te, a_tes, _test}
2. Jaccard Similarity of the document is:

$$JS_{Set\ ,Set2} = \frac{|\{Row2, Row3, Row4\}|}{|\{Row2, Row3, Row4, Row5\}|} = \frac{3}{4}$$

3. The minhash signatures for the following three orderings are shown:

ORDER	Shingle/Documents	Set 1	Set 2
1st	1	0	0
2nd	4	1	1
3rd	3	1	1
4th	5	1	0

5th	2	1	1
MH_1(Set n)	-	2nd	2nd

ORDER	Shingle/Documents	Set 1	Set 2
1st	4	1	1
2nd	1	0	0
3rd	2	1	1
4th	5	1	0
5th	3	1	1
MH_1(Set n)	-	1st	1st

ORDER	Shingle/Documents	Set 1	Set 2
1st	5	1	0
2nd	4	1	1
3rd	3	1	1
4th	2	1	1
5th	1	0	0
MH_1(Set n)	-	1st	2nd

4. The minhash signature matrix is:

Set 1	Set 2
2nd	2nd
1st	1st
1st	2nd

Similarity using signatures = 2/3

5. The locality sensitive hashing first divides the signature matrix into several horizontal bands and hashes each column using a hash function to hash buckets. The similarity is checked only for the documents that hash into the same bucket. It may be noted that the chances of similar document sets may get to the same hashed bucket by at least one hash function is high, whereas the documents that are not similar have low chances of hashing into the same bucket.

Check Your Progress 3:

1. Distance measures are used to find differences between two entities based on some criteria. They are used in problems that measure similarity, which is complementary to distance. The distance measure should be such that it is positive, symmetric and trigonometric (that is the distance between two points is less than or equal to the sum of the distance from the first point to a third point and the distance of the third point to the second point.)
2. Cosine distance measures the angular distance from a specific reference point. It is in general in the range from 0 to 180 degrees. Hamming distance is normally used as the binary distance between two Boolean vectors.
3. Some of the common uses of text analysis are the extraction of structured information from unstructured text, meaningful summarization of documents, question-answering systems and sentiment analysis.

5.9 REFERENCES/FURTHER READINGS

1. Leskovec J., Rajaraman R, Ullman J, **Mining of Massive Datasets,** 3rd Edition, available on the website http://www.mmds.org/
2. Gandomi A., Haider M., **Beyond the hype: Big data concepts, methods, and analytics**, International Journal of Information Management, Volume 35, Issue 2, 2015, Pages 137-144, ISSN 0268-4012.

UNIT 6 – OUTLIER DETECTION

6.0 INTRODUCTION

Outliers are data points that significantly differ from the majority of observations in a dataset. In predictive data analysis, they may arise due to measurement errors, data entry mistakes, or genuine but rare events. Understanding outliers is important because they can heavily influence statistical measures such as mean and standard deviation, and may distort patterns learned by predictive models, leading to inaccurate results. At the same time, not all outliers are errors—some may carry valuable insights, such as fraud detection, rare diseases, or unusual customer behaviour.

For example, in a credit card transaction dataset, most purchases may range between ₹100 and ₹5,000, but a sudden transaction of ₹2,00,000 could appear as an outlier. This unusual value might indicate fraudulent activity rather than normal spending behavior. Identifying such outliers helps financial institutions take timely action, improving both security and decision-making in predictive systems.

In Unit 5, *Similarity Measures*, we explored how data points can be compared based on distance, proximity, and resemblance using various mathematical metrics. These measures form the backbone of many predictive data analysis techniques, enabling tasks such as clustering, classification, and recommendation systems. However, an important challenge arises when certain observations do not conform to the general structure of the data. Such atypical observations—known as outliers—can significantly distort similarity computations, leading to misleading patterns and inaccurate predictive models.

Now, this Unit 6, *Outlier Detection*, is directly based upon the concepts of similarity by focusing on identifying and handling these anomalous data points. Techniques such as the standard deviation method, Z-score, and percentiles rely on statistical dispersion to detect

deviations from the norm, while box-plots provide intuitive visual mechanisms for spotting extreme values. Moving beyond purely statistical approaches, methods like DBSCAN and Local Outlier Factor (LOF) leverage similarity and density concepts introduced in this unit to detect outliers in complex, high-dimensional datasets. By understanding how outliers influence similarity relationships and model behaviour, this unit equips learners with essential tools to improve data quality, enhance model robustness, and ensure more reliable predictive analytics outcomes.

6.1 EXPECTED LEARNING OUTCOMES

- Understand the concept and significance of outliers in predictive data analysis.
- Apply the standard deviation method to identify deviations from the mean.
- Use Z-score to quantify how far a data point lies from the average.
- Interpret percentiles to detect extreme values within a dataset.
- Utilize box-plots for visual identification of outliers.
- Implement DBSCAN to detect noise and outliers based on data density.
- Analyse anomalies using the Local Outlier Factor (LOF) method.
- Evaluate the impact of outliers on similarity measures and predictive models.

6.2 STANDARD DEVIATION METHOD

The standard deviation method is one of the most widely used statistical techniques for identifying how much individual data points deviate from the mean (average) of a dataset. In the context of outlier detection, it helps us determine whether a data point lies unusually far from the central tendency of the data.

Standard deviation measures the spread or dispersion of data around the mean.

- A small standard deviation indicates that values are close to the mean.
- A large standard deviation indicates that values are widely spread out.

In outlier detection, observations that lie far beyond the typical spread (usually more than 2 or 3 standard deviations from the mean) are considered potential outliers. To understand this statement, we need to understand a few fundamentals like Mean, Variance etc. Their respective formulas and related context we explore few numerical given below:

Example: Consider the dataset representing daily sales (in ₹): 100, 110, 105, 98, 102, 500, does this data hint you for outlier, Check it.

Solution: We know the Formula for Mean (μ) $is\ given\ by\ \ \mu = (\sum xi)/N$

Consider the dataset representing daily sales (in ₹): 100,110,105,98,102,500; the mean (μ)

$$\mu = \{100 + 110 + 105 + 98 + 102 + 500\} / \{6\} = 169.17$$

The mean (μ) is ₹169.17, which is unusually high in comparison to most of the values in the given data set, and this is due to the extreme value (₹500), hinting at a possible outlier.

Next, we need to explore Variance and Its Relation to Standard Deviation, actually Variance measures the average squared deviation from the mean, and it is represented by the formula $\sigma^2=\sum(xi-\mu)^2 / N$. Whereas Standard deviation (σ) is simply the square root of variance (σ^2)

Now, let's explore Variance and Its Relation to Standard Deviation for Outlier detection.

Step 1: Compute deviations from mean (μ = 169.17)

Value (xi)	xi − μ	(xi − μ) ²
100	-69.17	4784.49
110	-59.17	3501.09
105	-64.17	4127.79
98	-71.17	5065.15
102	-67.17	4511.79
500	330.83	109448.69

Step 2: Compute Variance

$\sigma^2=(4784.49+3501.09+4127.79+5065.15+4511.79+109448.69)/6 =21906.50$

Step 3: Compute Standard Deviation $\sigma \approx 148.02$

Now, let's, analyse the results for Identifying Outliers Using Standard Deviation, the common rule for outlier detection is as follows:

- Values beyond $\mu \pm 2\sigma \rightarrow$ Possible outliers
- Values beyond $\mu \pm 3\sigma \rightarrow$ Strong outliers

Based on this said rule let's Calculate Range for Lower & Upper Bounds respectively:

- Lower bound ($\mu \pm 2\ \sigma$): $169.17-(2\times148.02) = -126.87$
- Upper bound ($\mu \pm 3\ \sigma$): $169.17+(2\times148.02) = 465.21$

The Interpretation of the Results consolidates that the Most data points (100, 110, 105, 98, 102) fall within the acceptable range. However, the value 500 exceeds the upper bound (465.21) hence identified as an outlier. The presence of ₹500 significantly inflated both the mean and standard deviation, indicating that extreme values can distort statistical measures. Detecting and handling such outliers is essential for building accurate predictive models.

So, we learned that the standard deviation method provides a systematic way to measure deviations from the mean and identify unusual observations. By understanding its relationship with variance and applying threshold rules (such as $\pm2\sigma$ or $\pm3\sigma$), analysts can effectively detect outliers and improve the reliability of predictive data analysis.

6.3 Z-SCORE

We learned about the role of standard deviation in outlier detection, in the section 6.2 above. Here in this section 6.3, we are going to discuss the next important characteristic for outlier detection i.e. Z-score.

The Z-score (or standard score) is a statistical measure that indicates how far a data point lies from the mean in terms of standard deviations. It standardizes data, allowing us to compare observations across different scales. In outlier detection, the Z-score is especially useful because it provides a clear numerical measure of extremeness—helping identify values that deviate significantly from the norm.

Z-score, represents a standardized value that indicates how many standard deviations a particular data point is away from the mean of a dataset. The Z-score is essentially a normalized form of deviation that expresses distance from the mean in units of standard deviation. This relationship makes it a powerful tool in predictive data analysis, especially for detecting outliers, comparing datasets, and standardizing variables for further modeling.

Before starting this section, we need to understand that the Z-score and standard deviation are intrinsically connected, as the Z-score is fundamentally derived using the standard deviation to measure how far a data point lies from the mean, and the formula for computation of Z-Score value is:

$$\mathbf{Z=(x-\mu) / \sigma}$$

Where:

- x = individual data point
- μ = mean of the dataset
- σ = standard deviation

Basic Understanding of Z-Score

- A Z-score = 0 → the data point is exactly at the mean.
- A positive Z-score → the value is above the mean.
- A negative Z-score → the value is below the mean.
- Larger absolute values (|Z|) indicate greater deviation.

The Conceptual Relationship between Standard deviation (σ) and Z-score (Z) are as follows:

- The Standard deviation (σ) measures the overall spread of a dataset—how much the data varies around the mean, and
- The Z-score (Z) uses this spread to determine the relative position of an individual data point.

In simple terms, Standard deviation (σ) describes the dataset, while Z-score describes a single observation within that dataset.

The key insight to the formula (Z=(x−μ) / σ) is that without standard deviation, Z-score cannot be computed, and without Z-score, interpreting standard deviation at the individual data point level becomes difficult. That means that they work together such that Standard deviation (σ) provides the scale and Z-score provides the position on that scale

The relationship between Z-Score & σ, becomes particularly important in identifying outliers:

- Standard deviation defines the expected spread.
- Z-score identifies whether a value lies within or outside this spread.

The Typical rule for identifying outliers' states following:

- $|Z| \geq 2 \rightarrow$ Possible outlier
- $|Z| \geq 3 \rightarrow$ Strong outlier

In practice, the following thresholds are commonly used:

- $|Z| < 2 \rightarrow$ Normal data point
- $2 \leq |Z| < 3 \rightarrow$ Potential outlier
- $|Z| \geq 3 \rightarrow$ Strong outlier

This rule is based on the empirical distribution of data (assuming approximate normality), where most values lie within ±3 standard deviations.

Thus, we can say that Z-score operationalizes standard deviation for practical decision-making.

Example: Consider the dataset (daily transaction amounts in ₹): 50, 55, 52, 48, 51, 300
Analyse the given data and determine the outliers using Z-Score

Solution:

Step 1: Calculate Mean (μ) = (50+55+52+48+51+300) / 6 = 556/6 = 92.67

Step 2: Calculate Standard Deviation (σ) ≈ 92.10

Step 3: Compute Z-Scores for Each Value

Value (x)	$Z = (x - \mu)/\sigma$	Interpretation
50	(50 − 92.67)/92.10 = -0.46	Close to mean
55	-0.41	Normal
52	-0.44	Normal
48	-0.48	Normal
51	-0.45	Normal
300	(300 − 92.67)/92.10 = 2.25	Potential outlier

Interpretation of Results tabulated above identifies that Most values have Z-scores close to 0, indicating they are near the mean. The value 300 has a Z-score of 2.25, which lies beyond ±2 hence, flagged as a potential outlier. It is to be noted that if a stricter threshold (±3) is used, then the value 300 may not be considered an extreme outlier, but still requires attention.

☞ Check Your Progress 1

Q1. What are outliers? Explain their significance in predictive data analysis.

……

.....

Q2. Explain the Standard Deviation method for detecting outliers.

.....

.....

Q3. Define Z-score and explain its role in outlier detection.

.....

.....

Q4. Differentiate between Standard Deviation method and Z-score method.

.....

.....

Q5. Explain the relationship between Standard Deviation and Z-score in outlier detection.

.....

.....

6.4 PERCENTILES

Quantiles are positional measures that divide a dataset into equal parts and provide a deeper understanding of the distribution of data. Unlike measures such as mean, quantiles focus on the relative position of observations, making them highly useful in identifying the spread, concentration, and presence of extreme values (outliers). *The most commonly used quantiles are quartiles, deciles, and percentiles. In this section we are going to discuss these Quantiles with their relevance to Outlier detection.*

Outlier detection is an important aspect of data analysis as extreme values can significantly distort results and lead to incorrect interpretations. In this context, positional measures such as quantiles, percentiles, deciles, and median play a crucial role because they are based on the relative position of data rather than its magnitude. This makes them robust and less sensitive to extreme values, unlike the arithmetic mean.

We had already discussed the concept of Deciles, Quantiles and median in unit-1, here we are going to extend our discussion on these parameters for the detection of outliers. Before starting the mathematical aspects of the Percentiles, Deciles, Quantiles and median; we need to understand their relevance in the context of outlier detection.

Quantiles provide the overall framework for understanding the distribution of data by dividing it into equal parts. They help in identifying how observations are spread across the dataset and enable the setting of threshold boundaries beyond which values may be considered unusual. Since quantiles focus on order rather than value size, they are particularly effective in detecting outliers in skewed distributions.

The **Median**, *which is the second quartile (Q_2),* acts as a stable central reference point in the dataset. It divides the data into two equal halves and is not influenced by extreme observations. This makes it highly useful for identifying the presence of outliers indirectly. For instance, a large difference between the mean and median often indicates the presence of extreme values or skewness in the data. Thus, the median provides a reliable baseline against which unusually high or low values can be compared.

Quartiles, which are specific types of quantiles, are the most widely used measures for direct outlier detection. *The first quartile (Q_1) and the third quartile (Q_3)* define the spread of the middle 50% of the data, known as the **interquartile range (IQR)**. The IQR is calculated as $Q_3 - Q_1$, and it forms the basis for identifying outliers using standard statistical rules. Any value lying below $Q_1 - 1.5 \times IQR$ or above $Q_3 + 1.5 \times IQR$ is considered an outlier. This method is highly reliable because it is not affected by extreme values and works well even when the data is skewed.

Deciles further divide the dataset into ten equal parts, providing a more detailed segmentation than quartiles. They are useful in identifying extreme segments such as the lowest 10% or highest 10% of observations. Values lying beyond the first decile (D_1) or the ninth decile (D_9) can be treated as potential outliers. Deciles are particularly useful in applications such as performance evaluation, income distribution analysis, and risk assessment, where identifying extreme groups is important.

Percentiles offer the most precise level of data segmentation by dividing the dataset into one hundred equal parts. They are widely used for fine-grained outlier detection, especially in large datasets. Common practice involves identifying values below the 5th percentile (P_5) or above the 95th or 99th percentile (P_{95} or P_{99}) as outliers. Percentiles are extensively applied in fields such as education, finance, and machine learning, where detecting extreme observations is critical for decision-making and model accuracy.

So, we learned that quantiles, median, deciles, and percentiles together provide a comprehensive and robust framework for outlier detection. The median offers a stable central reference, quartiles enable structured boundary-based detection through the IQR method, while deciles and percentiles allow increasingly finer identification of extreme observations. Because these measures are position-based and not influenced by extreme values, they are highly reliable tools for analysing data distributions and identifying outliers effectively.

Now, we elaborate the mathematical aspects of quantiles, median, deciles, and percentiles with their respective formulas and supporting numerical.

Quartiles divide the data into four equal parts. There are three quartiles: Q_1, Q_2 (median), and Q_3. Q_1 represents the value below which 25% of observations lie, Q_2 divides the data into two equal halves, and Q_3 represents the value below which 75% of observations lie.

We already learned in Unit-1 that Median is the Second Quartile (Q2) and it relates to the Central Reference Point i.e. The median divides the dataset into two equal halves. The Outlier detection on the basis of Median follows the rules given below:

- If mean >> median → presence of high-end outliers
- If mean << median → presence of low-end outliers
- If mean ≈ median → No significant outliers

The Example given below, clarifies these rules

Case 1: Mean >> Median (High-end Outliers)

Data: 2, 3, 4, 5, 6, 7, 50
Step 1: Calculate Mean: $Mean = \frac{2+3+4+5+6+7+5}{7} = \frac{77}{7} = 11$
Step 2: Calculate Median: Sorted data: 2, 3, 4, 5, 6, 7, 50 therefore Median = 5

Now, the Results are: Mean = 11 and Median = 5 i.e. Mean >> Median
So, it can be interpreted that the mean is much higher than the median because of the extreme value 50. This indicates the presence of a high-end outlier.

Case 2: Mean << Median (Low-end Outliers)

Data: 1, 20, 22, 24, 25, 26, 27
Step 1: Calculate Mean: $Mean = \frac{1+20+22+24+25+26+27}{7} = \frac{145}{7} \approx 20.71$
Step 2: Calculate Median: Sorted data: 1, 20, 22, 24, 25, 26, 27 therefore Median = 24

Now, the Results are: Mean ≈ 20.71 and Median = 24 i.e. Mean << Median

So, it can be interpreted that the mean is pulled down due to the very small value **1**.
This indicates the presence of a **low-end outlier**.

Case 3: Mean ≈ Median (No Significant Outliers)

Data:10, 12, 14, 15, 16, 18, 20
Step 1: Calculate Mean: $Mean = \frac{105}{7} = 15$
Step 2: Calculate Median: Median = 15
Now the Results are: Mean = 15 and Median = 15 i.e. Mean ≈ Median
So, it can be interpreted that there is no significant difference between mean and median.
This indicates no extreme outliers and a symmetrical distribution.

This method is simple but effective for **quick detection of skewness and outliers**.

Now, let's extend our discussion to **Interquartile Range (IQR) Method for Outlier Detection,** we know that for grouped data, the formula for quartiles is:

$$Q_j = L + \frac{(pcf)}{f} \times i \quad for\ j = 1,2,3$$

where
L = lower limit of quartile class
N = total frequency
pcf = preceding cumulative frequency
f = frequency of quartile class
i = class interval

To perform Outlier detection using Quartiles, we need to understand that the Q1, Q2 & Q3 are the most widely used for outlier detection, and the same is supported by the **Interquartile Range (IQR) Method, where** $IQR = Q3 - Q1$ and the Outliers are identified by using the rules:

$$Lower\ Bound = Q1 - 1.5 \times IQR Upper\ Bound = Q3 + 1.5 \times IQR$$

The quartile-based outlier rule says that any observation below "$Lower\ Bound = Q1 - 1.5 \times IQR$"
or above "$Upper\ Bound = Q3 + 1.5 \times IQR$" *will be treated as an outlier.*

Interquartile Range (IQR) Method is considered as the Most standard and reliable method because it Works well even for skewed data, and it is Used in box plots and data cleaning

Example: Determine the Inter Quartile Range (IQR) and comment on the existence of Outliers in the grouped data given below:

Class Interval	Frequency
0–10	5
10–20	9
20–30	14
30–40	8
40–50	4

Solution:
Total frequency, N = (5+9+14+8+4) = 40

Prepare the Cumulative frequency table:

Class Interval	Frequency	Cumulative Frequency
0–10	5	5
10–20	9	14
20–30	14	28
30–40	8	36
40–50	4	40

Calculation of Q1

$$Q_1 = L + \frac{(pcf)}{f} \times i Q_1 = 10 + \frac{(5)}{9} \times 10 Q_1 = 10 + \frac{5}{9} \times 10 = 10 + 5.56 = 15.56$$

So,

$$Q_1 = 15.56$$

Calculation of Q3

$$Q_3 = L + \frac{(pcf)}{f} \times iQ_3 = 30 + \frac{(28)}{8} \times 10Q_3 = 30 + \frac{2}{8} \times 10 = 30 + 2.5 = 32.5$$

So,

$$Q_3 = 32.50$$

Calculation of IQR

$$IQR = Q_3 - Q_1 IQR = 32.50 - 15.56 = 16.94$$

So,

$$IQR = 16.94$$

Lower Bound for Outlier Detection

$$Lower\ Bound = \ Q_1 - 1.5(IQR) 15.56 - 1.5(16.94) = 15.56 - 25.41 = -9.85$$

So,

$$Lower\ Bound = Q_1 - 1.5(IQR) = -9.85$$

Upper Bound for Outlier Detection

$$Upper\ Bound \ = Q_3 + 1.5(IQR) 32.50 + 1.5(16.94) = 32.50 + 25.41 = 57.91$$

So,

$$Upper\ Bound \ = \ Q_3 + 1.5(IQR) = 57.91$$

The Analysis of the Obtained results can be summarized as follows:
The quartile-based outlier rule says that any observation below Lower bound i.e. -9.85 or above Upper Bound i.e. 57.91 will be treated as an outlier. In this dataset, the class intervals range from 0 to 50, which means all observations fall within the acceptable range. Therefore, no outliers are detected using the quartile method.

This shows that the data is fairly balanced and does not contain any extreme unusually low or high values. Quartiles are useful for outlier detection because they are based on positional values and are not heavily affected by extreme observations. That is why the IQR method is considered a reliable method for detecting outliers, especially in skewed data.

Now, we are in the position to extend our discussion for Deciles and Percentiles. We begin with Deciles and finally we conclude with Percentiles.

Deciles are the Quantiles that divide the data into ten equal parts, denoted as D_1, D_2, …, D_9. Each decile represents 10% of the data distribution, providing a more detailed segmentation than quartiles.
For grouped data, the formula is:

$$D_k = L + \frac{(pcf)}{f} \times i \ \ for \ \ \ k = 1,2,\ldots,9$$

where,

- $D_k \rightarrow$ The k[th] decile (e.g., D_1, D_5, D_9)
- $k \rightarrow$ Decile number (1 to 9)
- $N \rightarrow$ Total number of observations (sum of frequencies)
- $\frac{kN}{10} \rightarrow$ Position of the k-th decile in the dataset
- $L \rightarrow$ Lower limit of the class interval containing the k-th decile
- $pcf \rightarrow$ Preceding cumulative frequency (cumulative frequency before the decile class)
- $f \rightarrow$ Frequency of the decile class
- $i \rightarrow$ Class interval width

Before using the deciles for outlier detection let's understand how to calculate them by using the formula given above.

Example: Determine the D_5 and comment on the existence of Outliers in the grouped data given below:

Class Interval	Frequency
0–10	5
10–20	9
20–30	14
30–40	8
40–50	4

Solution:

N = (5+9+14+8+4) = 40

Position of the k[th] decile in the dataset = $\frac{kN}{10}$ therefore,

$$D_5 = \frac{5N}{10} = \frac{5 \times 40}{10} = 20^{th} \ observation$$

From cumulative frequency:

Class	c.f
10–20	14
20–30	28

So D_5 lies in class 20–30

Using the formula: $D_k = L + \frac{(pcf)}{f} \times if or \ k = 1,2,\ldots,9$

We get, $D_5 = 20 + \frac{(14)}{14} \times 10 = 20 + \frac{6}{14} \times 10 = 20 + 4.29 = 24.29$

For Conceptual Understanding of Decile calculation, the formula

$D_k = L + \frac{(pcf)}{f} \times i \ \ for \ \ \ k = 1,2,\ldots,9$ works in two steps:

Step 1: Locate the Position: $\frac{kN}{10}$; This tells us the position of the decile in the ordered dataset.

Step 2: Interpolation within the Class: Since data is grouped, we assume values are evenly distributed within the class. The formula: $\frac{(pcf)}{f} \times i$ calculates how far inside the class the decile lies.

The formula finds the exact value of the decile within a class interval, It adjusts the lower limit (L) based on how deep the required observation lies in that class, Essentially, it is a **linear interpolation method**

Let's apply this understanding to perform Calculation of Deciles (D1)

$$\frac{1 \times 40}{10} = 4^{th}\ observation$$

The 4th observation lies in class 0–10.

$$D_1 = 0 + \frac{(0)}{5} \times 10 = 8 D_1 = 8$$

Similarly, we can calculate other deciles from D_1 to D_9 the values obtained are:
$D_1 = 8$, D_2=13.33, $D_3 = 17.78$, $D_4 = 21.43$, $D_5 = 24.29$, $D_6 = 27.14$, $D_7 = 30$; $D_8 = 35$, $D_9 = 40$

It is to be noted that the deciles divide the data into ten equal parts and help us understand how the observations are spread across the distribution. Here, the first decile is $D_1 = 8$, which means nearly 10% of the observations are below 8, and the ninth decile is $D_9 = 40$, which means nearly 90% of the observations are below 40.

For outlier detection using deciles, following are the rules :

- values far below **D1** may be treated as unusually low,
- values far above **D9** may be treated as unusually high.
- Further, the Values below D_1 or above D_9 may indicate extreme observations, and it is Useful in ranking (top 10%, bottom 10%).
- The K^{th} Decile (D_k) gives the value below which k×10% data lies the Formula uses the Position → $\frac{kN}{10}$ and Adjustment i.e. interpolation within class, and it converts grouped data into a precise positional value

It can be interpreted from the result $D_5 = 24.29$ that 50% of observations lie below 24.29 and 50% lie above 24.29.

In the given dataset, the observations are spread smoothly from 0 to 50 without any large jump or isolated extreme class. The decile values also increase gradually, which suggests that the distribution is fairly regular. Therefore, based on deciles, there is **no strong evidence of extreme outliers** in the data.

- About 10% of values lie below **8**
- About 50% of values lie below **24.29**
- About 90% of values lie below **40**

So, the data appears reasonably distributed, and **no serious outlier problem is indicated by the deciles**.

Finally, it's time to conclude this section with the understanding of Percentiles, they are the Quantiles that divide the data into one hundred equal parts and are denoted as P_1, P_2, …, P_{99}. They provide the most detailed insight into the distribution of data.
The formula for grouped data is:

$$P_k = L + \frac{(pcf)}{f} \times i for\ k = 1,2,\ldots,99$$

Again, ***the Conceptual Understanding of Percentile calculation***, the formula $D_k = L + \frac{(pcf)}{f} \times i \quad for \quad k = 1,2,\ldots,99$ works in two steps:

Step 1: Locate the Position: $\frac{kN}{100}$; This tells us the position of the percentile in the ordered dataset.

Step 2: Interpolation within the Class: Since data is grouped, we assume values are evenly distributed within the class. The formula: $\frac{(pcf)}{f} \times i$ calculates how far inside the class the percentile lies.

Example: Determine the P_{80} and comment on the existence of Outliers in the grouped data given below:

Class Interval	Frequency
0–10	5
10–20	9
20–30	14
30–40	8
40–50	4

Solution:

Step 1: Locate the Position (P_k): $\frac{kN}{100}$ => $P_{80} = \frac{80N}{100} = \frac{80\times40}{100} = 32^{nd}\ observation$

Step 2: Interpolation within the Class:

From cumulative frequency(**c.f.**):

Class	c.f
20–30	28
30–40	36

So P_{80} lies in class 30–40

$$P_{80} = 30 + \frac{(28)}{8} \times 10 = 30 + \frac{4}{8} \times 10 = 30 + 5 = 35$$

For outlier detection using percentiles, following are the rules:

- Values below P_5 implies low-end outliers
- Values above P_{95} or P_{99} implies high-end outliers

Percentiles are widely used in Exam rankings, Salary distribution, Machine learning preprocessing etc.

In this section we learned that the Quantiles, deciles, and percentiles provide a structured way to understand data distribution. Quartiles are most commonly used for outlier detection through the IQR method, while deciles and percentiles offer finer segmentation for identifying extreme observations. Since these measures are based on position rather than magnitude, they are robust and reliable even in skewed datasets, making them highly valuable tools in statistical analysis and decision-making.

These Quartiles are used to develop boxplots, we are going to discuss them in the next section of this unit.

6.5 BOX-PLOTS

The Box plots are widely used for outlier detection, skewness analysis, and spread visualization. Actually, Box Plot (or Box-and-Whisker Plot) is a graphical representation of data distribution that summarizes a dataset using five key statistics given below:

- Minimum
- First Quartile (Q1)
- Median (Q2)
- Third Quartile (Q3)
- Maximum

To construct a box plot, we first compute:

1. Minimum value
2. Q1 (25th percentile)
3. Median (Q2)
4. Q3 (75th percentile)
5. Maximum value

Then calculate:

$$IQR = Q3 - Q1$$

Outlier Detection Rule

$$Lower\ Bound = Q1 - 1.5 \times IQR Upper\ Bound = Q3 + 1.5 \times IQR$$

- Values outside these bounds are referred as Outliers
- Values within bounds are referred as Normal data

We already learned the calculation of these statistical parameters in the section 6.4 , given above.
Now let's discuss the Steps to Create a Box Plot :

1. Arrange data in ascending order
2. Find median (Q2)
3. Find Q1 and Q3
4. Compute IQR
5. Identify outliers using bounds
6. Draw:
    - A box from Q1 to Q3
    - A line inside box at median
    - Whiskers to min & max (excluding outliers)
    - Plot outliers as separate points

Example: Draw Box Plot for the Data: 5, 7, 8, 9, 10, 12, 13, 15, 18, 22, 50

Solution:

Step 1: Calculate Median (Q2) : Total observations = 11

$$Q2 = 6^{th}\ value = 12$$

Step 2: Q1 (Median of lower half) : Lower half: 5, 7, 8, 9, 10

$$Q1 = 8$$

Step 3: Q3 (Median of upper half) : Upper half: 13, 15, 18, 22, 50

$$Q3 = 18$$

Step 4: IQR: $= Q3 - Q1 = 18 - 8 = 10$

Step 5: Outlier Bounds

$$Lower\ Bound = 8 - 1.5(10) = 8 - 15 = -7 Upper\ Bound = 18 + 1.5(10) = 18 + 15 = 33$$

Step 6: Identify Outliers

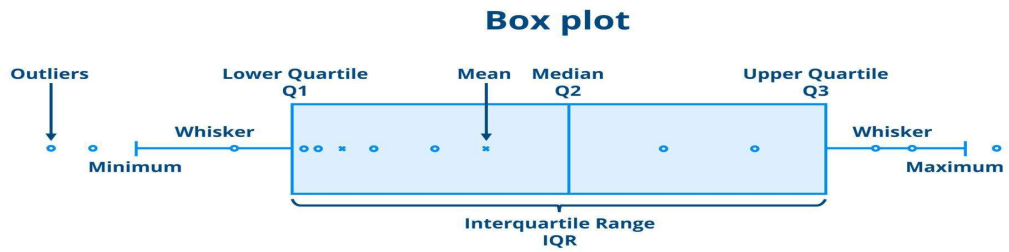
- Data values > 33 i.e. 50 is an outlier
- No lower outlier (since min = 5 > -7)

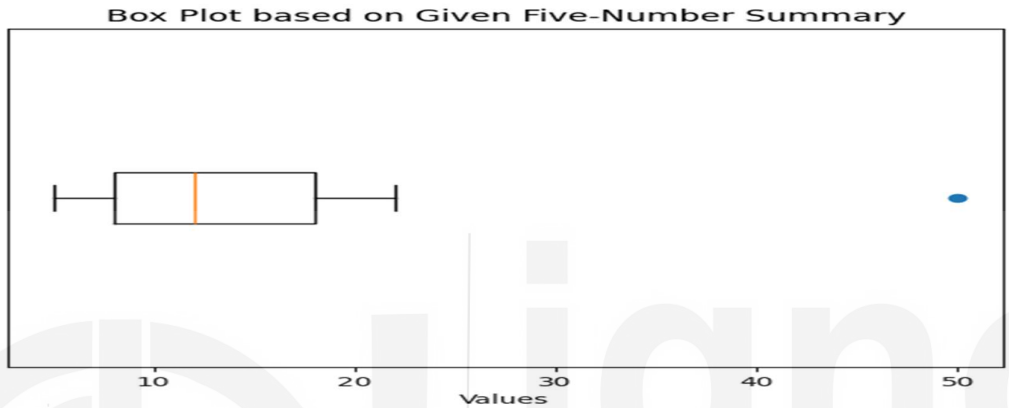
Final Five-Number Summary

- Minimum = 5
- Q1 = 8
- Median = 12
- Q3 = 18
- Maximum (without outlier) = 22
- Outlier = 50

Interpretation of the Box Plot

The box (8 to 18) shows middle 50% of data
The median (12) divides data into two halves
The whiskers (5 to 22) show normal range
The point 50 lies far beyond the upper bound
→ outlier





It can read from the Box plot that Left whisker is Minimum = 5, the Box starts at Q1 = 8 followed by a Middle line (Q2) i.e. Median = 12 and the Box ends at Q3 = 18. The Right whisker is Maximum (without outlier) = 22 and Point on far right is the Outlier = 50.

It can be Interpreted from the Box plot that the central 50% of data lies between 8 and 18 ; and the Median (12) is closer to Q1slightly right skewness (Data is right-skewed because of high outlier 50) and the value 50 is clearly separated, confirming it as a high-end outlier

We learned the preparation of Box plots, which is an important tool for Outlier Detection. It clearly highlights extreme values visually and uses IQR, a robust measure, not affected by outliers. It Helps in Data cleaning, Fraud detection, Statistical modelling, Comparing multiple datasets

Also, Box plots are also considered as a simple yet powerful tool to summarize data distribution, detect outliers using IQR, understand skewness and spread. In practice, box plots with the IQR method are among the most reliable techniques for outlier detection in statistics and data science.

We learned to draw Box Plot for the Discrete data now let's understand ***how to create the Box plot for the grouped data***

We learned that a Box-and-Whisker Plot visually displays the distribution, spread, and skewness of a dataset through five summary statistics:

- Minimum
- First Quartile (Q1)

- Median (Q2)
- Third Quartile (Q3)
- Maximum

Example: Construct the Box Plot for the Grouped data given below:

Class Interval	Frequency	Midpoint
48–53	4	50.5
54–59	6	56.5
60–65	6	62.5
66–71	6	68.5
72–77	6	74.5
78–83	5	80.5
84–89	5	86.5
90–95	2	92.5

Solution:
Now, *to create the Box plot for the grouped data, we need to follow following steps*

Step 1: Reconstruct the Raw Data from Grouped Data
Since a Box Plot needs raw data or an approximation of it, reconstruct a pseudo dataset using the midpoints of each class interval and the corresponding frequency. Here's how:

Class Interval	**Frequency**	**Midpoint**
48–53	4	50.5
54–59	6	56.5
60–65	6	62.5
66–71	6	68.5
72–77	6	74.5
78–83	5	80.5
84–89	5	86.5
90–95	2	92.5

Total observations (N) = 40

Construct Pseudo Raw Data: i.e. expand the data based on the frequency:

$50.5 \times 4 \rightarrow$ positions 1–4
$56.5 \times 6 \rightarrow$ positions 5–10
$62.5 \times 6 \rightarrow$ positions 11–16
$68.5 \times 6 \rightarrow$ positions 17–22
$74.5 \times 6 \rightarrow$ positions 23–28
$80.5 \times 5 \rightarrow$ positions 29–33
$86.5 \times 5 \rightarrow$ positions 34–38

92.5 × 2 → positions 39–40

Step 2: Arrange the Data in Ascending Order

Arrange the values from smallest to largest. Since the data is already binned using midpoints, and each midpoint is repeated according to its frequency, it's already structured to represent an ordered dataset.

Step 3: Find the Five-Number Summary i.e. Min., Max, Q1, Q2, and Q3

Using the pseudo raw data given above:

- Minimum: Smallest value = 50.5
- Maximum: Largest value = 92.5
- Median (Q2): Middle value (20th and 21st observation average) ≈ 68.5
- First Quartile (Q1): 25th percentile (10th observation) ≈ 58.0
- Third Quartile (Q3): 75th percentile (30th observation) ≈ 80.5

So, the five-number summary is:

- Min = 50.5
- Q1 = 58.0
- Median (Q2) = 68.5
- Q3 = 80.5
- Max = 92.5

Let's understand how to determine the Quartiles Q1, Q2 and Q3; especially in the context of a dataset like the one we've been working with (40 student marks). You can use the same method for any dataset.

So, What Are Quartiles? we have 3 quartiles in general Q1, Q2, and Q3; their respective meaning is as follows:

- Q1 (First Quartile): The value that separates the lowest 25% of the data.
- Q2 (Median): The middle value; separates the lower 50% from the upper 50%.
- Q3 (Third Quartile): The value that separates the lowest 75% from the top 25%.

Now, we understand How to Determine Q1, Q2 and Q3 (Manually)

Assume you have N = 40 data points, arranged in ascending order.

Step A: Find the Positions

- Q1 position = $(N+1)/4=(41)/4=10.25^{th}$ item
- Q2 position = $(N+1)/2 = (41)/2 = 20.5^{th}$ item
- Q3 position = $3(N+1)/4 =3(41)/4 =30.75^{th}$ item

These are positions, not values yet.

Step B: Estimate the Quartile Values

To get the values at 10.25th and 30.75th positions, you interpolate between the actual values at the 10th, 11th (for Q1), 20th, 21st (for Q2), and 30th, 31st (for Q3).

Suppose the ordered pseudo data looks like:

(Refer to Pseudo Raw Data given above)

50.5 × 4 → positions 1–4

56.5 × 6 → positions 5–10
62.5 × 6 → positions 11–16
68.5 × 6 → positions 17–22
74.5 × 6 → positions 23–28
80.5 × 5 → positions 29–33

Finding Quartile-1(Q1), from the pseudo raw data given we can see that there are
4 values of 50.5
6 values of 56.5 → positions 5 to 10
6 values of 62.5 → positions 11 to 16

So, the 10th value = 56.5, and 11th value = 62.5
Thus, Q1=56.5+0.25 × (62.5−56.5) = 56.5+1.5=58.0

(Q2) Median position=(N+1)/2=41/2=20.5th value
We want the value at the 20.5th position in the ordered pseudo data.

Now from the pseudo raw data given we can see that both
20^{th} value = 68.5 and 21^{st} value = 68.5
Thus, Q2 = (68.5+68.5)/2=68.5
Since the 20.5th value lies between 68.5 and 68.5, the median is 68.5

Similarly, 30th value = 80.5, 31st value = 86.5
Q3=80.5+0.75 × (86.5−80.5) = 80.5+4.5=85.0

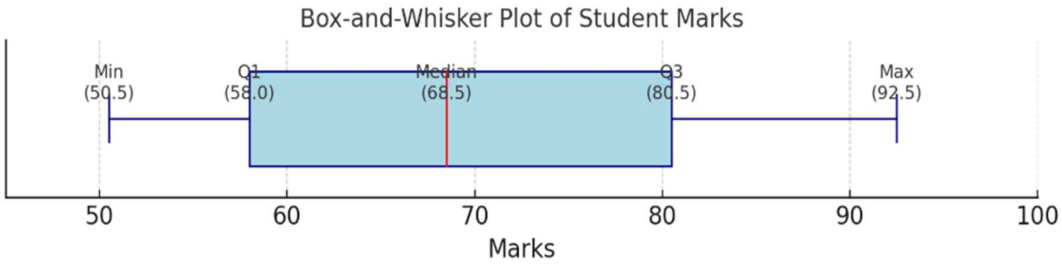
So, Q1 ≈ 58.0, Q2 ≈ 68.5 and Q3 ≈ 85.0 in this case.

Step 4: Draw the Box-and-Whisker Plot

- Draw a horizontal number line that covers the range of your data (e.g., from 45 to 100).
- Mark the five-number summary values on the line.
- Draw a rectangular box from Q1 to Q3 .
- Draw a vertical line inside the box at the median .
- Draw whiskers (horizontal lines) from:
    - Min to Q1
    - Q3 to Max

Step 5: Label the Plot

- Label the five-number summary on the axis.
- Title your plot appropriately: *"Box-and-Whisker Plot of Student Marks"*



Here is the Box-and-Whisker Plot based on the student marks data. It visually displays the five-number summary:

Minimum: 50.5 ; Q1 (First Quartile): 58.0 ; Median (Q2): 68.5 ;
Q3 (Third Quartile): 80.5 ; Maximum: 92.5

This plot helps you understand the spread, centre, and symmetry of the data, as well as spot potential outliers (though none appear here).

In short, the box plot helps to:

- Visualize the spread and central tendency.
- Detects symmetry or skewness.
- Identify potential outliers (values far from the whiskers).
- Quickly compare multiple datasets if needed.

We learned that the Box plots provide a deeper look into data distribution and help identify variability and outliers.

☞ Check Your Progress 2

Q1. What are Percentiles and how are they used in outlier detection?

.....

.....

Q2. Explain the Interquartile Range (IQR) method for detecting outliers.

.....

.....

Q3. What is the relationship between Quartiles and Box Plots?

.....

.....

Q4. Differentiate between Percentile-based and IQR-based outlier detection methods.

.....

.....

Q5. Why are Box Plots considered effective for outlier detection and data visualization?

.....

.....

Q6. What is a Box Plot? Explain its components.

.....

.....

Q7. Explain how outliers are detected using Box Plots.

.....

.....

Q8. Describe the steps to construct a Box Plot.

.....

.....

Q9. Interpret a Box Plot in terms of skewness and spread.

.....

.....

Q10. Why are Box Plots considered a reliable method for outlier detection?

.....

.....

6.6 DBSCAN

DBSCAN stands for Density-Based Spatial Clustering of Applications with Noise. It is a density-based clustering algorithm used to identify groups of closely packed data points and, at the same time, detect points that do not belong to any dense region. These isolated points are treated as outliers or noise. DBSCAN is especially useful when the data contains irregularly shaped clusters and when outlier detection is an important objective.

Unlike partition-based methods such as K-means, DBSCAN does not require the number of clusters to be specified in advance. Instead, it uses two parameters, namely Eps and MinPts, to determine whether a region is dense enough to form a cluster. Because of this density-based logic, DBSCAN is widely used in anomaly detection, fraud analysis, geographic data analysis, image processing, and spatial mining.

The main idea of DBSCAN is simple: if a point lies in a sufficiently dense neighbourhood, then it belongs to a cluster. If a point lies alone or in a sparse region, then it is treated as noise or an outlier.

DBSCAN classifies points into three categories:

- **Core Point:** A point is called a **core point** if at least **MinPts** points lie within its neighbourhood of radius **Eps**.
  Where,
  a. **Condition for Core Point is:** A point p is a core point if:
  $$| N_\varepsilon(p) | \geq MinPts$$

where $| N_\varepsilon(p) |$ means the number of points in the Eps-neighbourhood of p, usually including the point itself.

b. **Eps:** Eps, written as ε, is the maximum distance within which points are considered neighbours.

c. **Eps-Neighbourhood:** For a point p, the Eps-neighbourhood is defined as:

$$N_\varepsilon(p) = \{q \in D \mid dist(p,q) \leq \varepsilon\}$$

where:

- D= dataset
- $dist(p,q)$= distance between points $p$and q

This means all points within distance $\varepsilon$from point $p$are its neighbours.

d. **MinPts:** MinPts is the minimum number of points required in the Eps-neighbourhood for a point to be considered a core point.

- **Border Point:** A point is called a **border point** if it lies within the Eps-neighbourhood of a core point, but it does not itself have enough neighbouring points to become a core point.
- **Noise Point / Outlier:** A point is called **noise** if it is neither a core point nor a border point. Such points do not belong to any cluster and are treated as outliers.

Important Note:

Practical Meaning of Parameters in Outlier Detection

- **If Eps is too small:** Many points may be marked as outliers because neighbourhoods become too small.
- **If Eps is too large:** Different clusters may merge, and true outliers may get absorbed into clusters.
- **If MinPts is too high:** Small but valid clusters may be treated as noise.
- **If MinPts is too low:** Noise points may incorrectly become clusters.

So, parameter tuning is essential.

The figure below describes Core point, Border Point and Noise points.

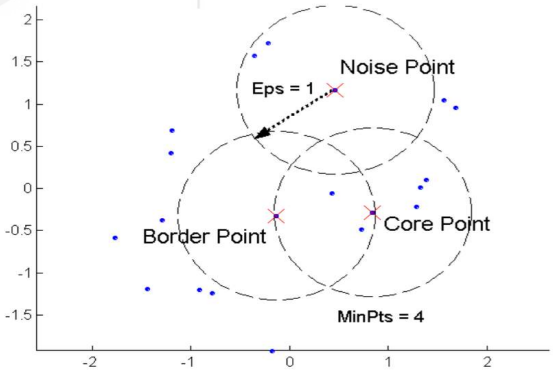


Figure – 6.1: Core point, Border Point and Noise points.

The most common distance used in DBSCAN is Euclidean distance:

$$dist(p,q) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

For higher-dimensional data: $dist(p,q) = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2}$

Key Concepts Used in DBSCAN are:

- **Directly Density-Reachable:** A point $q$is directly density-reachable from point p if: $q \in N_\varepsilon(p)$, and $p$is a core point
- **Density-Reachable:** A point is density-reachable from another point if there exists a chain of directly density-reachable points connecting them.
- **Density-Connected:** Two points are density-connected if there exists some point from which both are density-reachable.

These ideas allow DBSCAN to form clusters of arbitrary shape.

DBSCAN Algorithm: The working of DBSCAN can be described in the following steps:

Step 1: Select an unvisited point from the dataset.
Step 2: Find all points in its Eps-neighbourhood.
Step 3: If the number of neighbours is at least MinPts, mark it as a core point and start a new cluster.
Step 4: Add all density-reachable points to this cluster. If any of these points are also core points, expand the cluster further.
Step 5: If a point does not satisfy the core-point condition and is not reachable from any core point, mark it as noise.
Step 6: Repeat the process until all points are visited.

Example: Apply DBSCAN algorithm on the following two-dimensional dataset: $A(1,1),\ B(1,2),\ C(2,1),\ D(2,2),\ E(8,8),\ F(8,9),\ G(9,8),\ H(25,25)$ *to identify the possible clusters,* using Euclidean distance. Given $\varepsilon = 1.5, MinPts = 3$

Solution:

Step 1: Check neighbourhood of A (1,1):

Therefore, Distances from A:

$$dist(A,B) = \sqrt{(1-1)^2 + (2-1)^2} = 1 dist(A,C) = \sqrt{(2-1)^2 + (1-1)^2} = 1 dist(A,D) = \sqrt{(2-1)^2 + (2-1)^2} = \sqrt{2} \approx 1.41$$

All are within $\varepsilon = 1.5$

So,$N_\varepsilon(A) = \{A, B, C, D\}$ i.e. Number of points = 4, Since $| N_\varepsilon(A) | = 4 \geq 3$ thus, A is a **core point**.

Step 2: Check neighbourhood of B (1,2)

Therefore, Distances from B:

$$dist(B,A) = 1$$

$$dist(B, C) = \sqrt{2} \approx 1.41$$

$$dist(B, D) = 1$$

So, $N_\varepsilon(B) = \{A, B, C, D\}$Again, 4 points. So, B is also a **core point**.
Similarly, C and D will also have enough neighbours within distance 1.5.Thus, points **A, B, C, D** form **Cluster 1**.

Step 3: Check neighbourhood of E(8,8)

Therefore, Distances from E:

$$dist(E, F) = 1 \qquad dist(E, G) = 1$$

$$dist(E, H) = \sqrt{(25-8)^2 + (25-8)^2} = \sqrt{578} \approx 24.04$$

So,$N_\varepsilon(E) = \{E, F, G\}$, Number of points = 3, Since $| N_\varepsilon(E) | = 3 \geq 3$, E is a **core point**.

Now check F and G: $dist(F, G) = \sqrt{(9-8)^2 + (8-9)^2} = \sqrt{2} \approx 1.41$
So each of E, F, and G lies close to the others. Hence **E, F, G** form **Cluster 2**.

Step 4: Check neighbourhood of H (25,25)

Distances of H from all other points are much larger than 1.5. So, $N_\varepsilon(H) = \{H\}$, Number of points = 1, Since $| N_\varepsilon(H) | = 1 < 3$, H is **not** a core point. Also, H is not within the Eps-neighbourhood of any core point. Therefore, **H is a noise point**, that is, an **outlier**.

Final Result of Clusters

- Cluster 1: $\{D\}$
- Cluster 2: $\{G\}$
- Outlier / Noise H (25,25)

It can be interpreted from the Point of View of Outlier Detection that In this example, DBSCAN successfully identifies two dense groups of points and also detects one point, $H(25,25)$, as an outlier. This happens because H is far away from all dense regions and does not have enough neighboring points within radius $\varepsilon = 1.5$.

This interpretation is important because of following reasons:

- points A, B, C, and D are in a dense local region, so they form one cluster
- points E, F, and G form another dense local region
- point H is isolated, so DBSCAN marks it as noise

Thus, from an outlier detection perspective, DBSCAN is very effective because it uses **local density** rather than global averages or fixed cluster assignments.

Summary Table

Point	Eps-Neighbourhood Size	Type	Cluster
A	4	Core	Cluster 1
B	4	Core	Cluster 1
C	4	Core	Cluster 1
D	4	Core	Cluster 1
E	3	Core	Cluster 2
F	3	Core/Border depending on counting	Cluster 2
G	3	Core/Border depending on counting	Cluster 2
H	1	Noise / Outlier	None

Advantages of DBSCAN

- No need to specify number of clusters in advance
- Detects outliers naturally
- Can find clusters of arbitrary shape
- Works well with noisy data
- Useful in spatial and anomaly-detection problems

Limitations of DBSCAN

- Choice of $\varepsilon$and MinPts is very important
- Struggles when cluster densities vary a lot
- Performance can reduce in high-dimensional data
- Sensitive to scale of variables, so normalization may be needed

We learned from the above example that *DBSCAN is particularly powerful for outlier detection* because it does not force every point into a cluster. Any point lying in a sparse region automatically becomes noise. This is a major advantage over methods like K-means, where every point is assigned to some cluster, even if it is abnormal.

From the outlier detection point of view, DBSCAN:

- identifies isolated points naturally
- works well when outliers are far from dense groups
- does not assume normal distribution
- can detect clusters of arbitrary shape
- is robust to noise

This section concludes with the understanding that DBSCAN is a powerful clustering and outlier detection algorithm based on the concept of density. It forms clusters where data points are closely packed and labels isolated points as noise. The method is especially valuable in real-world datasets where cluster shapes are irregular and outlier detection is important. In the numerical example, DBSCAN correctly formed two clusters and identified one isolated point as an outlier. This clearly demonstrates how the algorithm supports anomaly detection in a natural and intuitive way.

The final topic in this unit is the local Outlier factor; we are going to discuss it in next section 6.7 given below.

6.7 LOCAL OUTLIER FACTOR

The **Local Outlier Factor (LOF)** is a powerful density-based technique used in anomaly detection to identify observations that deviate significantly from their local neighbourhood. Unlike traditional global outlier detection methods, which evaluate a data point against the entire dataset, LOF focuses on the *local structure* of the data. This makes it particularly useful in real-world datasets where data is not uniformly distributed, and different regions exhibit varying densities.

The fundamental idea behind LOF is intuitive yet mathematically grounded, where a data point is considered an outlier if its **local density is substantially lower than that of its surrounding neighbours**. In other words, LOF does not simply label points far from the centre as anomalies; instead, it detects points that are isolated *relative to their immediate context*. For instance, a point in a sparse region may not be an outlier if its neighbours are equally sparse, whereas a point in a dense cluster that is relatively isolated from nearby points would be flagged as anomalous.

To understand how LOF works, it is essential to explore its key concepts and definitions. The method begins by defining the **k-nearest neighbours (k-NN)** of each data point. For a given point, these are the k closest points based on a chosen distance metric, typically Euclidean distance. This neighbourhood forms the basis for evaluating local density.

Next, LOF introduces the concept of **reachability distance**, which adjusts the raw distance between two points to account for the density of the neighbour. Specifically, the reachability distance between a point *A* and its neighbour *B* is defined as the maximum of two quantities: the actual distance between *A* and *B*, and the distance from *B* to its nearest neighbour. This adjustment ensures that points in dense regions do not appear artificially close to points in sparser regions, thereby stabilizing density comparisons.

Building on this, the **local reachability density (LRD)** of a point is computed. This represents how densely the point is surrounded by its neighbours and is defined as the inverse of the average reachability distance from the point to its k-nearest neighbours. A higher LRD indicates that the point lies in a dense region, while a lower LRD suggests a sparse neighbourhood.

Finally, the **Local Outlier Factor (LOF)** score is calculated by comparing the local reachability density of a point with the densities of its neighbours. Specifically, it is the average ratio of the LRDs of the point's neighbours to the LRD of the point itself. This ratio quantifies how isolated the point is relative to its surroundings. A LOF value close to 1 indicates that the point has a density similar to its neighbours and is therefore considered normal. A value significantly greater than 1 indicates that the point has a much lower density than its neighbours and is likely an outlier. Conversely, values less than 1 suggest that the point lies in a denser region than its neighbours.

So we learned that LOF provides a nuanced and context-aware approach to anomaly detection by evaluating the relative density of data points within their local neighbourhoods. This makes it highly effective in identifying subtle anomalies in complex datasets, especially where clusters of varying densities exist.

A brief introduction of the Key Concepts with their Formulas and Definitions are as follows:

(a) k-distance: For a point p, the k-distance is the distance to its **k-th nearest neighbor**.

For any data point p, the *k-distance* is defined as the distance between p and its nearest neighbor. This value determines the radius of the neighbourhood around the point. It is important to note that if multiple points are at the same distance, more than k neighbours may be included. The choice of k plays a crucial role: a small k captures very local structures, while a larger k provides a more global perspective. The *k-distance* of a point p is defined as the distance between p and its nearest neighbour in the dataset. Formally, it can be expressed as:

k-distance(p)=dist(p,ok)

where:

- dist(p,ok) is the distance between point p and its kth nearest neighbour ok,
- ok represents the kth nearest neighbour of p.

If we assume a standard distance metric such as Euclidean distance, then:

$$\text{k-distance}(p) = \sqrt{\sum_{i=1}^{d}(p_i - o_{k,i})^2}$$

where:

- d is the number of dimensions,
- pi and $o_{k,i}$ are the coordinates of p and its k^{th} neighbor in the i^{th} dimension.

This formula identifies the radius that defines the local neighbourhood around point p, which is then used in subsequent LOF calculations.

(b) k-distance Neighbourhood

The *k-distance neighbourhoods* of a point p, denoted as $N_k(p)$, includes all points whose distance from p is less than or equal to the k-distance. It is formally expressed as:

$$N_k(p) = \{dist(p,q) \leq k - distance(p)\}$$

This includes all points within the k-distance.

This neighbourhood defines the local region around the point p that will be used for density estimation. It may contain more than k points due to ties in distance.

(c) Reachability Distance: The *reachability distance* is a modified distance measure that improves the robustness of density estimation. It is defined as:

$$reach - dist_k(p,q) = max\ (k - distance(q),\ dist(p,q))$$

This definition ensures that distances smaller than the k-distance of the neighbor qqq are adjusted upward. The key purpose is to prevent extremely small distances from disproportionately increasing the density estimate, especially in very dense clusters. In effect, it smooths the influence of neighbours and stabilizes the computation. It prevents very small distances from dominating density calculations.

(d) Local Reachability Density (LRD)

The *local reachability density* of a point p, denoted as LRD(p), measures how densely the point is surrounded by its neighbours. It is computed as the inverse of the average reachability distance:

$$LRD(p) = \frac{| N_k(p) |}{\sum_{q \in N_k(p)} reach - dist_k(p, q)}$$

- $| N_k(p) |$ = how many neighbours
- $\sum_{q \in N_k(p)} reach - dist_k(p, q)$ = total distance to neighbours (adjusted via reach-dist)

$| N_k(p) |$ means Total number of k-nearest neighbours of point p. It is NOT the value of q; It is NOT the sum of neighbours. It is simply the count of neighbours

So, LRD is basically: "Number of neighbours per unit distance" i.e. **LRD** Represents how densely point p is surrounded.

A higher LRD value indicates that the point lies in a dense region (small average reachability distance), whereas a lower LRD suggests that the point is in a sparse region. Thus, LRD serves as a local density estimator.

(e) Local Outlier Factor (LOF)

The *Local Outlier Factor* of a point p, denoted as LOF(p), quantifies how much the density of ppp deviates from the densities of its neighbours. It is defined as:

$$LOF(p) = \frac{\sum_{q \in N_k(p)} \frac{LRD(q)}{LRD(p)}}{| N_k(p) |}$$

This formula computes the average ratio of the local reachability densities of the neighbours to that of the point p. If p has a density similar to its neighbours, the ratio will be close to 1. If p has a significantly lower density, the LOF value will be greater than 1, indicating a potential outlier.

these concepts i.e. k-distance, k-distance Neighbourhood, Reachability distance, Local Reachability Density (LRD) and Local Outlier Factor (LOF) work together in a structured manner: the k-distance defines the neighbourhood, the reachability distance stabilizes pairwise distances, the LRD estimates local density, and the LOF score compares densities to identify anomalies. This layered formulation allows LOF to effectively detect local outliers even in datasets with varying density distributions.

Interpretation of LOF Values: After the calculation of Local Outlier Factor (LOF), the LOF values can be interpreted as follows:

LOF Value	Interpretation
≈ 1	Normal point
> 1	Possible outlier
>> 1	Strong outlier

The Local Outlier Factor (LOF) algorithm follows a systematic sequence of steps to identify anomalous data points based on their local density. Understanding each step clearly helps you to grasp how the method works in practice.

Step 1: Choose parameter k (number of neighbours)
The first step is to select the value of k, which determines how many nearest neighbours will be considered for each data point. This parameter controls the "locality" of the analysis. A smaller k focuses on very close neighbours and captures fine-grained patterns, while a larger k provides a broader view of the data structure. Choosing an appropriate k is important because it directly affects the sensitivity of outlier detection.

Step 2: Compute k-distance for each point
For every data point p, calculate the distance to its kth nearest neighbour. This value, called the *k-distance*, defines the radius of the neighbourhood around p. It helps establish how far we need to look to include the nearest k points.

Step 3: Find k-nearest neighbours
Using the k-distance, identify the set of neighbours $N_k(p)$ for each point. These are all points whose distance from p is less than or equal to the k-distance. This step forms the local neighbourhood that will be used to estimate density.

Step 4: Compute reachability distance
Next, compute the *reachability distance* between each point p and its neighbours q. This adjusted distance takes into account both the actual distance and the density around the neighbour. It ensures that very small distances do not overly influence the density calculation, making the method more stable and reliable.

Step 5: Compute Local Reachability Density (LRD)
For each point p, calculate its *local reachability density (LRD)*. This is essentially the inverse of the average reachability distance between p and its neighbours. If the average distance is small, the density is high; if the average distance is large, the density is low. Thus, LRD quantifies how densely the point is surrounded by other points.

Step 6: Compute LOF score
Now, compute the *Local Outlier Factor (LOF)* for each point by comparing its LRD with the LRDs of its neighbours. This step measures how different the density of a point is from the densities of nearby points. A value close to 1 indicates that the point has a similar density to its neighbours, while a value significantly greater than 1 indicates a potential outlier.

Step 7: Identify outliers
Finally, analyse the LOF scores to identify anomalies. Points with high LOF values (typically greater than 1) are considered outliers because they are located in regions of lower density compared to their neighbours. The higher the LOF score, the stronger the indication that the point is an outlier.

Finally, the Steps of LOF Algorithm are:

1. Choose parameter **k (number of neighbours)**
2. Compute **k-distance** for each point
3. Find **k-nearest neighbours**
4. Compute **reachability distance**

5. Compute **LRD for each point**
6. Compute **LOF score**
7. Identify points with **high LOF as outliers**

The LOF algorithm works by gradually building from neighbourhood identification to density estimation and finally to density comparison. This step-by-step approach allows it to effectively detect local anomalies even in complex datasets with varying densities, making it a powerful and widely used technique in data analysis.

Example: Determine the Outliers by applying Local Outlier Factor (LOF) algorithm on the 1-Dimensional Dataset: $1,\ 2,\ 2,\ 3,\ 3,\ 4,\ 10$ $Given\ k = 2$

Solution: Let us apply the Local Outlier Factor (LOF) algorithm step-by-step on the given 1D dataset:

Dataset: {1,2,2,3,3,4,10}, with k=2

Step 1: Sort the Data

It is observed that the given data is already sorted: 1,2,2,3,3,4,10

Step 2: Compute k-distance and k-nearest neighbours

For k=2, we find the 2 nearest neighbours for each point:

Point	2-Nearest Neighbours	k-distance
1	2, 2	1
2(1)	1, 2	1
2(2)	2, 3	1
3(1)	2, 3	1
3(2)	3, 4	1
4	3, 3	1
10	4, 3	6

Let's understand how the 2-Nearest Neighbours are identified

Given Dataset (1D): {1,2,2,3,3,4,10}, and k=2

Since the data is 1D, the distance between two points is simply: dist (a, b) = |a−b|

Now, to Compute k-distance and k-nearest neighbours for each point follow the steps:

1. Compute distance to all other points
2. Sort distances in ascending order
3. Pick the 2 nearest neighbours (k = 2)
4. The distance to the 2nd nearest neighbor = k-distance

1. For point = 1 in the Given Dataset (1D): {1,2,2,3,3,4,10}, and k=2

Distances: |1−2|=1,|1−2|=1,|1−3|=2,|1−3|=2,|1−4|=3,|1−10|=9
Sorted distances are: 1,1,2,2,3,9
Among the Sorted Distances analyse the first two entries i.e. |1−2|=1,|1−2|=1; the 2 nearest neighbours are 2 & 2, and the k-distance is 1. So, 2 nearest neighbours are (2,2) and the k-distance = 1

Similarly, for other points, as shown below:

2. For point = 2 (first occurrence)

Distances: |2−1|=1, |2−2|=0, |2−3|=1, |2−3|=1, |2−4|=2, |2−10|=8

Sorted: 0,1,1,1,2,8 ; with 2 nearest neighbours→2,1*(including duplicate 2) and* k-distance=1

Next,

3. For point = 2 (second occurrence)

Distances: |2−2|=0, |2−3|=1, |2−3|=1, |2−1|=1, |2−4|=2, |2−10|=8

Sorted: 0, 1, 1, 1, 2, 8; with 2 nearest neighbours → 2,3 and k-distance = 1

4. For point = 3 (first occurrence)

Distances: |3−2|=1, |3−2|=1, |3−3|=0, |3−4|=1, |3−1|=2, |3−10|=7

Sorted: 0, 1, 1, 1, 2, 7 with 2 nearest neighbours → 3,2 and k-distance = 1

5. For point = 3 (second occurrence)

Distances: |3−3|=0, |3−4|=1, |3−2|=1, |3−2|=1, |3−1|=2, |3−10|=7

Sorted: 0, 1, 1, 1, 2, 7 with 2 nearest neighbours → 3,4 and k-distance = 1

6. For point = 4

Distances: |4−3|=1, |4−3|=1, |4−2|=2, |4−2|=2, |4−1|=3, |4−10|=6

Sorted: 1, 1, 2, 2, 3, 6 with 2 nearest neighbours → 3,3 and k-distance = 1

7. For point = 10

Distances: |10−4|=6, |10−3|=7, |10−3|=7, |10−2|=8, |10−2|=8, |10−1|=9

Sorted: 6, 7, 7, 8, 8, 9 with 2 nearest neighbours → 4,3 and k-distance = 7 (distance to 2nd nearest neighbor)

Final Summary Table

Point	**2-Nearest Neighbours**	**k-distance**
1	2, 2	1
2	2, 1	1
2	2, 3	1
3	3, 2	1
3	3, 4	1
4	3, 3	1
10	4, 3	7

Important Insights: From the above table it can be interpreted that all cluster points have small k-distance (≈1) but the Point 10 has a large k-distance (7) and this is the early indication of outlier.

Now, we are done with **Step 2: Compute k-distance and k-nearest neighbours; of the** Local Outlier Factor (LOF) algorithm. The next step is to Determine the Reachability Distance.

Step 3: Compute Reachability Distance:

$$reach - dist_k(p,q) = max\ (k - distance(q),\ dist(p,q))\ ;$$

where $dist(p,q) = |\ p - q\ |$

Now compute for key points:

- For point 1 (neighbours: 2, 2): reach-dist = max(1,1) = 1 for both neighbours → avg = 1
- For point 2 / 3 / 4 (cluster points): distances are small → reach-dist mostly = 1
- For point 10 (neighbours: 4, 3):

- reach-dist(10,4) =max(1,6)=6
- reach-dist(10,3) =max(1,7)=7

Now, the next step of the Local Outlier Factor (LOF) algorithm is to determine the Local Reachability Density (LRD)

Step 4: Compute Local Reachability Density (LRD)

$$LRD(p) = \frac{| N_k(p) |}{\sum_{q \in N_k(p)} reach - dist_k(p,q)}$$

- $| N_k(p) |$ = how many neighbours
- $\sum_{q \in N_k(p)} reach - dist_k(p,q)$ = total distance to neighbours (adjusted via reach-dist)

$| N_k(p) |$ means Total number of k-nearest neighbours of point p. It is NOT the value of q; It is NOT the sum of neighbours. It is simply the count of neighbours

So, LRD is basically: "Number of neighbours per unit distance" i.e. **LRD** Represents how densely point p is surrounded.

A higher LRD value indicates that the point lies in a dense region (small average reachability distance), whereas a lower LRD suggests that the point is in a sparse region. Thus, LRD serves as a local density estimator.

Refer to Summary table derived above under Step-2, as per the data available for point p = 1

For p=1, k=2 Identified Nearest neighbours of 1 are 2 and 2. So: $| N_k(1) |$ ={2,2}. Thus,$| N_k(1) |$ = 2 Because there are 2 neighbors. Now, q represents each individual neighbour and It is used to calculate the denominator part of LRD i.e. $\sum_{q \in N_k(p)} reach - dist_k(p,q)$. Take each neighbour q, Compute reach-distance and add them.

For p=1 we got $| N_k(1) |$*=2 and reach-dist(1,2) + reach-dist(1,2) = 1 + 1 = 2 ;*
Thus, LRD(1)=2/(1+1) = 2/2 = 1

Similarly calculate LRD For cluster points (1,2,2,3,3,4): LRD ≈ 2/ (1+1) = 1

*For point p = 10 (**Outlier candidate**) the Neighbours are 4, 3.*

- For q = 4: d (10,4) = 6 and k-distance (4) = 1. Also, reach-dist = max (1,6) = 6
- For q = 3: d (10,3) = 7 and k-distance (3) = 1. Also, reach-dist = max (1,7) = 7

Therefore, LRD (10) = 2 / (6+7) =2 / 13 ≈ 0.154

Now the final step is to determine Local Outlier Factor (LOF)

Step 5: Compute Local Outlier Factor (LOF)

$$LOF(p) = \frac{\sum_{q \in N_k(p)} \frac{LRD(q)}{LRD(p)}}{| N_k(p) |}$$

LOF for point p = 1; Neighbors → 2, 2; Apply LOF formula: LOF (1) = {(1/1) + (1/1)} / 2 = 1

So, LOF (1) = 1 i.e. it is a Normal point

Similarly, LOF for point p = 4**;** Neighbors → 3, 3; LOF(4)= {(1/1) + (1/1)} / 2 = 1
So, LOF (4) = 1 is also a Normal point

Now, LOF for point p = 10; Neighbors → 4 and 3; LRD(4) = 1 ; LRD(3) = 1;and LRD(10) = 0.154
Therefore, LOF(10) = {(1/0.154)+(1/0.154)} / 2 = 6.49

- For cluster points: 1,2,2,3,3,4 LOF=1
- For point 10: Neighbours have LRD(4) = 1 ; LRD(3) = 1;and LRD(10) = 0.154
  Thus, LOF (10) = (1/0.154+1/0.154) / 2 = (6.49+6.49) / 2 ≈ 6.49

After the calculation of Local Outlier Factor (LOF), the LOF values can be interpreted as follows:

LOF Value	Interpretation
≈ 1	Normal point
> 1	Possible outlier
>> 1	Strong outlier

Thus, it can be interpreted from the results that Points 1,2,2,3,3,4 have LOF = 1 and they are the Normal points in the dataset. However, LOF for Point 10 is 6.49 hence identified as a Strong Outlier

Final Conclusion from the above results can be expressed as: The LOF algorithm clearly identifies 10 as an outlier because its local density is much lower compared to its neighbours. All other points form a dense cluster and are considered normal.

Finally, we understood that LOF is a density-based outlier detection method that compares the local density of a point with that of its neighbours. If a point has significantly lower density than its surroundings, it is identified as an outlier. In the given example, point 10 has a very high LOF value, confirming it as a strong outlier. Thus, LOF is highly effective for detecting anomalies in datasets with varying density patterns.

We learned the working of Local Outlier Factor (LOF) algorithm for Outlier detection. Now, we are going to conclude this section with a brief on the Advantages and Limitations of Local Outlier Factor (LOF) Algorithm

Advantages of Local Outlier Factor (LOF)

1. **Captures local anomalies**
   LOF focuses on the *local neighbourhood* of each data point rather than the entire dataset. This means it can identify points that are unusual in their immediate surroundings, even if they may not appear extreme globally. This makes it very effective for detecting subtle outliers.

2. **Works well in multi-density datasets**
   In real-world data, different regions may have different densities. Traditional methods may fail in such cases, but LOF adapts by comparing a point only with its nearby neighbors. This allows it to correctly identify outliers in both dense and sparse regions.

3. **More flexible than distance-based methods**
   Unlike simple distance-based techniques that rely only on how far a point is from others, LOF

considers *relative density*. This added flexibility helps in identifying complex patterns where distance alone is not sufficient.

4. **Useful in real-world applications**
   LOF is widely used in domains like fraud detection, intrusion detection, and medical diagnosis because it can detect unusual behavior patterns, rare events, or abnormal observations effectively.

Limitations of Local Outlier Factor (LOF)

1. **Sensitive to parameter k**
   The choice of the number of neighbors (k) significantly affects the results. A very small or very large k can lead to incorrect detection of outliers, so selecting an appropriate value is important and sometimes challenging.

2. **Computationally expensive for large datasets**
   LOF requires computing distances between points and their neighbors, which becomes time-consuming as the dataset size increases. This makes it less efficient for very large datasets without optimization.

3. **Performance depends on distance metric**
   The results of LOF depend heavily on how distance is measured (e.g., Euclidean distance). If the chosen distance metric does not properly reflect similarity in the data, the results may be misleading.

4. **Struggles in high-dimensional data**
   In very high-dimensional spaces, distances between points tend to become similar (a problem known as the *curse of dimensionality*). This reduces the effectiveness of LOF, as it becomes harder to distinguish between normal points and outliers.

☞ Check Your Progress 3

Q1. What is DBSCAN? Explain its role in outlier detection.

.....

.....

Q2. Define the key parameters of DBSCAN.

.....

.....

Q3. Differentiate between core, border, and noise points in DBSCAN.

.....

.....

Q4. What are the advantages of DBSCAN in outlier detection?

.....

.....

Q5. What is the Local Outlier Factor (LOF)?

.....

.....

Q6. Explain how LOF detects outliers.

.....

.....

Q7. Differentiate between DBSCAN and LOF.

.....

.....

Q8. Why are density-based methods preferred for complex datasets?

.....

.....

Q9. What are the limitations of DBSCAN?

.....

.....

Q10. Explain the importance of LOF in real-world applications.

.....

.....

6.8 SUMMARY

Outlier Detection is an essential aspect of data analysis that focuses on identifying unusual or extreme observations that significantly differ from the rest of the dataset. These outliers may arise due to errors, variability, or rare events, and their identification is crucial for improving data quality and model accuracy. One of the simplest approaches is the Standard Deviation method, where data points lying far from the mean (typically beyond ±2 or ±3 standard deviations) are considered outliers. Similarly, the Z-score method standardizes values and identifies outliers based on how many standard deviations a data point is away from the mean. Percentiles provide another intuitive approach by flagging values that fall in extreme ranges (e.g., below the 5th percentile or above the 95th percentile). These statistical techniques are widely used due to their simplicity and effectiveness in normally distributed data.

Graphical techniques such as Box-plots also play an important role in outlier detection by visually representing data distribution through quartiles. Values lying beyond the whiskers (usually 1.5 times the interquartile range) are considered outliers. Moving beyond basic methods, advanced techniques such as DBSCAN (Density-Based Spatial Clustering of Applications with Noise) identify outliers as points that do not belong to any dense cluster, making it effective for detecting anomalies in complex datasets. Similarly, the Local Outlier Factor (LOF) method evaluates the local density of a data point compared to its neighbors, identifying points that have significantly lower density as outliers.

Overall, the methods covered in this Unit, provides a comprehensive toolkit for detecting anomalies in both simple and complex datasets. While statistical methods are effective for structured and normally distributed data, advanced techniques like DBSCAN and LOF are more suitable for high-dimensional and non-linear data. By combining these approaches, analysts can ensure accurate detection of outliers, leading to more reliable data analysis and better decision-making.

6.9 ANSWERS

☞ Check Your Progress 1

Q1. What are outliers? Explain their significance in predictive data analysis.
Solution: Refer to Section 6.0
Q2. Explain the Standard Deviation method for detecting outliers.
Solution: Refer to Section 6.2
Q3. Define Z-score and explain its role in outlier detection.
Solution: Refer to Section 6.3
Q4. Differentiate between Standard Deviation method and Z-score method.
Solution: Refer to Section 6.2 & 6.3
Q5. Explain the relationship between Standard Deviation and Z-score in outlier detection.
Solution: Refer to Section 6.2 & 6.3

☞ Check Your Progress 2

Q1. What are Percentiles and how are they used in outlier detection?
Solution: Refer to Section 6.4
Q2. Explain the Interquartile Range (IQR) method for detecting outliers.
Solution: Refer to Section 6.4
Q3. What is the relationship between Quartiles and Box Plots?
Solution: Refer to Section 6.4
Q4. Differentiate between Percentile-based and IQR-based outlier detection methods.
Solution: Refer to Section 6.4
Q5. Why are Box Plots considered effective for outlier detection and data visualization?
Solution: Refer to Section 6.5
Q6. What is a Box Plot? Explain its components.
Solution: Refer to Section 6.5
Q7. Explain how outliers are detected using Box Plots.
Solution: Refer to Section 6.5
Q8. Describe the steps to construct a Box Plot.
Solution: Refer to Section 6.5
Q9. Interpret a Box Plot in terms of skewness and spread.
Solution: Refer to Section 6.5
Q10. Why are Box Plots considered a reliable method for outlier detection?
Solution: Refer to Section 6.5

☞ **Check Your Progress 3**

Q1. What is DBSCAN? Explain its role in outlier detection.
Solution: Refer to Section 6.6
Q2. Define the key parameters of DBSCAN.
Solution: Refer to Section 6.6
Q3. Differentiate between core, border, and noise points in DBSCAN.
Solution: Refer to Section 6.6
Q4. What are the advantages of DBSCAN in outlier detection?
Solution: Refer to Section 6.6
Q5. What is the Local Outlier Factor (LOF)?
Solution: Refer to Section 6.7
Q6. Explain how LOF detects outliers.
Solution: Refer to Section 6.7
Q7. Differentiate between DBSCAN and LOF.
Solution: Refer to Section 6.6 & 6.7
Q8. Why are density-based methods preferred for complex datasets?
Solution: Refer to Section 6.6
Q9. What are the limitations of DBSCAN?
Solution: Refer to Section 6.6
Q10. Explain the importance of LOF in real-world applications.
Solution: Refer to Section 6.7

6.10 REFERENCES/FURTHER READINGS

1. *Data Mining: Concepts and Techniques* by Jiawei Han, Micheline Kamber, and Jian Pei, published by Morgan Kaufmann (3rd Edition, 2011, ISBN: 978-0123814791)
2. *An Introduction to Statistical Learning with Applications in R by Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani, published by Springer (2nd Edition, 2021, ISBN: 978-1071614174).*
3. *Practical Statistics for Data Scientists by Peter Bruce, Andrew Bruce, and Peter Gedeck, published by O'Reilly Media (2nd Edition, 2020, ISBN: 978-1492072942).*
4. *Business Analytics: Data Analysis and Decision Making* by S. Christian Albright and Wayne L. Winston, published by Cengage Learning (7th Edition, 2020, ISBN: 978-0357131787).
5. *Introduction to Operations Research* by Frederick S. Hillier and Gerald J. Lieberman, published by McGraw-Hill Education (10th Edition, 2014, ISBN: 978-0073523453).
6. *Decision Analysis for Management Judgment* by Paul Goodwin and George Wright, published by Wiley (5th Edition, 2014, ISBN: 978-1119974017).
7. *Handbook of Statistical Analysis and Data Mining Applications* by Robert Nisbet, John Elder, and Gary Miner, published by Elsevier (2nd Edition, 2017, ISBN: 978-0124166455).
8. *Data Mining and Analysis: Fundamental Concepts and Algorithms* by Mohammed J. Zaki and Wagner Meira Jr., published by Cambridge University Press (1st Edition, 2014, ISBN: 978-0521766333).

9. *Core Concepts in Data Analysis: Summarization, Correlation and Visualization* by *Boris Mirkin, published by Springer (1st Edition, 2011, ISBN: 978-0857292872).*
10. *Applied Predictive Modeling, Max Kuhn and Kjell Johnson, 1st Edition, Springer, 2013.*
11. *An Introduction to Statistical Learning with Applications in R, Gareth James, Daniela Witten, Trevor Hastie and Robert Tibshirani, 1st Edition, Springer, 2013.*
12. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction, Trevor Hastie, Robert Tibshirani and Jerome Friedman, 2nd Edition, Springer, 2009.*
13. *An Introduction to Statistical Learning with Applications in R by Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani, published by Springer (2nd Edition, 2021, ISBN: 978-1071614174).*
14. *Time Series Analysis: Forecasting and Control by George E. P. Box, Gwilym M. Jenkins, Gregory C. Reinsel, and Greta M. Ljung, published by Wiley (5th Edition, 2015, ISBN: 978-1118675021)*

UNIT 7 - PREDICTIVE ANALYTICS MODELS

7.0 INTRODUCTION

Predictive Analytics Models represent a crucial stage in the data analytics continuum, where the focus shifts from understanding past patterns to forecasting future outcomes. This unit introduces the foundational models and techniques that enable organizations and researchers to anticipate trends, behaviours, and events based on historical data. By leveraging statistical methods and machine learning approaches, predictive analytics transforms raw data into forward-looking insights that support proactive decision-making in domains such as business, healthcare, finance, and technology.

The unit begins with an introduction to regression models, which are used to establish relationships between dependent and independent variables and predict continuous outcomes such as sales, demand, or risk. It then explores time series and forecasting models, which analyse data over time to identify trends, seasonal patterns, and cyclical variations, enabling accurate forecasting of future values. These models are particularly important in areas like financial forecasting, inventory management, and economic analysis.

Further, the unit covers supervised learning models, where algorithms are trained on labelled data to make predictions or classifications, and unsupervised learning models, which identify hidden patterns, structures, or groupings in unlabelled data. While supervised learning focuses on prediction with known outcomes, unsupervised learning helps in discovering unknown relationships and insights. Together, these models provide a comprehensive framework for predictive analytics, equipping learners with the necessary tools to build intelligent systems and make data-driven predictions in real-world scenarios.

7.1 EXPECTED LEARNING OUTCOMES

After completing this unit, you will be able to:

- Understand the fundamentals of predictive analytics and its role in forecasting future outcomes.
- Explain the concepts and applications of regression models for prediction of continuous variables.
- Analyse time series data and apply forecasting techniques for trend and pattern identification.
- Differentiate between and apply supervised and unsupervised learning models.
- Develop the ability to use predictive models for data-driven decision-making in real-world scenarios.

7.2 REGRESSION MODELS - AN INTRODUCTION

Regression analysis is a fundamental statistical technique used to understand and model the relationship between variables. In simple terms, it helps us study how a dependent variable (the outcome we want to predict) changes when one or more independent variables (the factors influencing it) change. The primary objective of regression is not only to identify this relationship but also to use it for predicting future outcomes based on known data.

Mathematically, regression can be expressed in a general form as:

$$Y = f(X_1, X_2, \ldots, X_n) + \varepsilon$$

where Yrepresents the dependent variable, $X_1, X_2, \ldots, X_n$represent independent variables, and $\varepsilon$is the error term capturing the variation not explained by the model. This equation shows that the value of Ydepends on the function of the input variables along with some unavoidable randomness.

To understand this better, consider a simple example where a student's marks depend on the number of hours studied. Suppose the relationship is modelled as:

$$Y = 10 + 8X$$

Here, Yrepresents marks and Xrepresents study hours. If a student studies for 5 hours, then:

$$Y = 10 + 8(5) = 50$$

This means the predicted marks are 50, illustrating how regression helps in prediction.

A key concept in regression is distinguishing between dependent and independent variables. The dependent variable is the output we want to predict, such as salary, marks, or sales, while independent variables are the inputs influencing it, such as experience, study hours, or advertising expenditure. For instance, consider a salary model:

$$Salary = 20000 + 5000 \times Experience$$

If a person has 3 years of experience, the predicted salary becomes:

$$Salary = 20000 + 5000(3) = 35000$$

This clearly demonstrates how regression quantifies the relationship between variables.

The most commonly used form of regression is the linear regression model, which assumes a straight-line relationship between variables. It is represented as:

$$Y = \beta_0 + \beta_1 X + \varepsilon$$

where $\beta_0$is the intercept and $\beta_1$is the slope, indicating how much Ychanges for a unit change in X. For example, if:

$$Y = 2 + 3X$$

and $X = 4$, then:

$$Y = 2 + 12 = 14$$

This shows that for every unit increase in X, Yincreases by 3 units.

However, relationships between variables are not always linear. Regression can also capture non-linear relationships. For example:

$$Y = 2 + X^2$$

If $X = 3$, then:

$$Y = 2 + 9 = 11$$

This represents a curved relationship. Additionally, relationships can be positive or negative. A positive relationship means both variables increase together, such as:

$$Y = 4X$$

while a negative relationship means one increases while the other decreases, such as:

$$Y = 20 - 2X$$

If $X = 5$, then:

$$Y = 10$$

Regression analysis is widely used across various domains. In finance, it helps predict stock prices. For example:

$$Price = 100 + 5X$$

If the market index is 10, then:

$$Price = 150$$

In healthcare, regression can estimate disease risk based on factors like age. For example:

$$Risk = 0.5 + 0.1X$$

If age is 40:

$$Risk = 4.5$$

Similarly, in marketing, it helps forecast sales:

$$Sales = 1000 + 50 \times Ads$$

If 10 advertisements are run:

$$Sales = 1500$$

These examples show how regression transforms raw data into meaningful predictions.

Regression also plays a central role in predictive analytics. It allows organizations to make data-driven decisions by identifying key factors influencing outcomes and estimating future values. For instance, if a model is:

$$Y = 3X + 2$$

and $X = 6$, then:

$$Y = 20$$

This simple prediction mechanism is the foundation of many real-world applications.

It is important to distinguish regression from other analytical techniques. Regression predicts continuous numerical values, whereas classification predicts categories (e.g., spam or not spam). Correlation only measures the strength of association between variables, not prediction, and time series analysis focuses specifically on time-dependent data. For example, while regression might predict house prices, classification would categorize houses as “cheap” or “expensive.”

Despite its usefulness, regression analysis comes with several challenges. One major issue is overfitting, where the model becomes too complex and fits the training data too closely, reducing its ability to generalize. For example, a model like:

$$Y = X^5 + X^4 + X^3$$

may fit the data perfectly but fail on new data. Another challenge is multicollinearity, where independent variables are highly correlated, such as height in centimeters and inches, which can make the model unstable.

Outliers are another concern. Consider the dataset: 10, 12, 11, and 100. The mean becomes:

$$\bar{X} = 33.25$$

which is misleading due to the extreme value 100. Missing data also poses a problem, as incomplete observations reduce the reliability of the model. Additionally, regression models rely on assumptions such as linearity, independence, and normality, and violations of these assumptions can lead to incorrect conclusions.

We are having a variety of **Regression models** *like Linear Regression, Multiple regression, polynomial regression Ridge and Lasso regression, also Support Vector regression*. In this section we will discuss these models in brief. However, the *details will be available in Unit-8*.

The Overview of the regression models with respective examples is as follows:

1. Linear Regression

Linear regression is the most basic and widely used regression technique, which models the relationship between a dependent variable and a single independent variable using a straight line. It assumes that the dependent variable changes at a constant rate with respect to the independent variable. This makes it simple to interpret and very useful for many real-world prediction problems.

The mathematical form of linear regression is given by:

$$Y = \beta_0 + \beta_1 X + \varepsilon$$

where $\beta_0$represents the intercept, $\beta_1$represents the slope (rate of change), and εrepresents the error term. The slope indicates how much the dependent variable changes for a one-unit increase in the independent variable.

For example, consider predicting the price of a house based on its area. Suppose the regression model is:

$$Price = 500000 + 1000 \times Area$$

If the area of the house is 100 square feet, then:

$$Price = 500000 + 1000(100) = 600000$$

This means that for every additional square foot, the price increases by ₹1000, clearly demonstrating a linear relationship.

Important Note:

- **Linear Regression** is the simplest regression technique and is best suited for situations where there is a **single independent variable** and the relationship between the variables is approximately **linear (straight-line)**. It is highly interpretable & computationally efficient, making it ideal for basic predictive tasks and initial data analysis. However, it cannot capture complex or non-linear relationships and is sensitive to issues like outliers and multicollinearity (though the latter is less relevant with a single variable).
- **When to use:** Linear regression should be used when the dataset is simple, the relationship between variables is linear, and interpretability is important. For example, predicting salary based only on years of experience.

2. Multiple Regression

Multiple regression extends the concept of linear regression by including more than one independent variable. In real-life situations, outcomes are rarely influenced by a single factor, so multiple regression helps capture the combined effect of several variables on the dependent variable.

The general equation for multiple regression is:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_n X_n + \varepsilon$$

Here, each independent variable contributes separately to the prediction of the dependent variable.

For example, suppose we want to predict salary based on both experience and education level. The model could be:

$$Salary = 20000 + 5000 \times Experience + 10000 \times Education$$

If a person has 3 years of experience and an education level of 2, then:

$$Salary = 20000 + 5000(3) + 10000(2) = 55000$$

This shows how multiple factors combine to influence the outcome, making predictions more realistic.

Important Note:

- **Multiple Regression** extends linear regression to include **multiple independent variables**, allowing it to model more realistic scenarios where several factors influence the outcome. While it improves prediction accuracy compared to simple linear regression, it introduces challenges such as **multicollinearity**, where independent variables are highly correlated.
- **When to use:** Multiple regression is appropriate when the dependent variable is influenced by **more than one predictor**, and the relationships remain approximately linear. For instance, predicting house prices based on area, number of rooms, and location.

3. Polynomial Regression

Polynomial regression is used when the relationship between variables is not linear but follows a curved pattern. Instead of fitting a straight line, it fits a curve by including higher powers of the independent variable.

The general form of polynomial regression is:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \ldots + \beta_n X^n$$

This allows the model to capture more complex relationships.

For instance, consider modelling sales growth over time where growth accelerates rather than increasing at a constant rate. Suppose the model is:

$$Sales = 100 + 10X + 2X^2$$

If $X = 3$, then:

$$Sales = 100 + 30 + 18 = 148$$

This shows that as time increases, sales increase at an increasing rate, indicating a non-linear trend.

Important Note:

- **Polynomial Regression** is used when the relationship between variables is **non-linear but still follows a structured mathematical curve**. It enhances linear models by introducing higher-degree terms (such as X^2, X^3), enabling the model to fit curved patterns. However, it can easily lead to **overfitting** if the degree of the polynomial is too high.
- **When to use:** Polynomial regression should be used when data shows a **clear curved trend** that cannot be captured by a straight line but still follows a predictable pattern. For example, modelling population growth or sales trends that accelerate over time.

4. Ridge Regression

Ridge regression is an advanced technique used to address overfitting, which occurs when a model becomes too complex and performs poorly on new data. Ridge regression introduces a penalty term to the model that reduces the magnitude of coefficients, making the model simpler and more stable.
The objective function for Ridge regression is:

$$Minimize: \sum(Y_i - \hat{Y}_i)^2 + \lambda \sum \beta_j^2$$

Here, $\lambda$is a tuning parameter that controls the strength of the penalty.

For example, consider predicting house prices using multiple features such as area, number of rooms, and location. A normal regression model might be:

$$Price = 500000 + 3000X_1 + 7000X_2 + 12000X_3$$

After applying Ridge regression, the coefficients are reduced:

$$Price = 500000 + 2500X_1 + 6000X_2 + 10000X_3$$

This reduction helps prevent overfitting and improves model performance on unseen data.

Important Note:

- **Ridge Regression** is a regularized version of multiple regression that adds an **L2 penalty** to reduce the magnitude of coefficients. This helps address issues like **overfitting and multicollinearity** by shrinking coefficients without eliminating any features. It improves model stability when dealing with many correlated predictors.
- **When to use:** Ridge regression is suitable when the dataset contains **many features with high multicollinearity**, and you want to **retain all variables** while reducing overfitting. It is commonly used in scenarios like financial modeling or high-dimensional datasets.

5. Lasso Regression

Lasso regression is another regularization technique similar to Ridge regression, but with an important difference: it can reduce some coefficients to exactly zero. This means it not only prevents overfitting but also performs feature selection by removing less important variables.
The objective function for Lasso regression is:

$$Minimize: \sum(Y_i - \hat{Y}_i)^2 + \lambda \sum | \beta_j |$$

In a real-world example, suppose we are predicting sales using several factors:

$$Sales = 1000 + 50X_1 + 30X_2 + 10X_3$$

After applying Lasso regression, the model might become:

$$Sales = 1000 + 50X_1 + 0X_2 + 10X_3$$

Here, the coefficient of X_2becomes zero, meaning that this feature is no longer relevant. This simplifies the model and improves interpretability.

Important Note:

- **Lasso Regression** is another regularization technique that uses an **L1 penalty**, which not only shrinks coefficients but can also reduce some of them to exactly zero. This makes it useful for **automatic feature selection**, simplifying the model and improving interpretability.
- **When to use:** Lasso regression should be used when you have **many features and suspect that some are irrelevant**, and you want the model to automatically select the most

important variables. It is particularly useful in domains like marketing analytics or biomedical data analysis where feature selection is critical.

6. Support Vector Regression (SVR)

Support Vector Regression is a powerful technique used for modeling complex and non-linear relationships. Unlike traditional regression methods, SVR does not try to minimize the error for all data points. Instead, it fits a model within a margin of tolerance (called epsilon) and ignores errors that fall within this margin.

The basic function of SVR is:

$$f(x) = wX + b$$

The optimization objective is:

$$Minimize\ \frac{1}{2} ||\ w\ ||^2$$

subject to:

$$|\ Y - f(X)\ | \leq \epsilon$$

This means the model allows small errors (within ϵ) and focuses on capturing the overall trend.
For example, consider predicting stock prices where there is always some noise in the data. Suppose the model is:

$$Y = 100 + 5X$$

If $X = 10$, then:

$$Y = 150$$

If the actual value is 152 and the tolerance $\epsilon = 5$, then this error is ignored because it lies within the acceptable range. This makes SVR robust to noise and suitable for real-world data.

Important Note:

- **Support Vector Regression (SVR)** is a more advanced method that works well for **complex and non-linear relationships**. Instead of minimizing the error for all data points, SVR tries to fit a function within a specified margin (epsilon), ignoring small errors and focusing on overall trends. It can handle non-linearity effectively using **kernel functions**, but it is computationally more intensive and less interpretable.
- **When to use:** SVR is best used when the data is **complex, non-linear, and possibly noisy**, and traditional regression methods fail to capture the underlying pattern. It is commonly applied in areas such as stock price prediction, demand forecasting, and other real-world problems with irregular patterns.

We learned that each regression model serves a different purpose depending on the nature of the data. Linear regression is useful for simple relationships, multiple regression handles multiple influencing factors, polynomial regression captures non-linear patterns, Ridge and Lasso help prevent overfitting, and Support Vector Regression is suitable for complex and noisy datasets. Together, these models form a comprehensive toolkit for predictive data analysis.

The section concludes with the understanding that the regression analysis is a powerful and widely used tool for modelling relationships and making predictions. By understanding how variables interact and

applying appropriate mathematical models, regression enables better decision-making across fields such as finance, healthcare, marketing, and engineering. However, careful attention must be given to model assumptions and limitations to ensure accurate and reliable results.

☞ **Check Your Progress 1**

Q1. What is Regression Analysis? Explain its objective in predictive analytics.

.....

.....

Q2. Differentiate between dependent and independent variables with examples.

.....

.....

Q3. Explain Linear Regression with its mathematical form and example.

.....

.....

Q4. What are the different types of regression models? Briefly explain any two.

.....

.....

Q5. Discuss the challenges in regression analysis.

.....

.....

7.3 TIME SERIES & FORECASTING MODELS – AN INTRODUCTION

Time series analysis is a specialized form of data analysis that focuses on observations collected over time. Unlike general statistical models that treat observations independently, time series models recognize that past values influence future values. This makes them particularly useful for forecasting, where the goal is to predict future outcomes based on historical data.

Mathematically, a time series can be represented as:

$$Y_t = f(t) + \varepsilon_t$$

where $Y_t$is the value at time t, $f(t)$represents the underlying pattern such as trend or seasonality, and $\varepsilon_t$is the random error component. This equation highlights that every observed value is a combination of structured patterns and randomness.

A time series is generally composed of four main components, which can be expressed as:

$$Y_t = T_t + S_t + C_t + R_t$$

Here, $T_t$represents the long-term trend, $S_t$represents seasonal variations, $C_t$represents cyclical fluctuations, and $R_t$represents random noise. Understanding these components is essential for building accurate forecasting models.

For example, consider monthly sales data where the trend component is 100 units, the seasonal effect adds 20 units, the cyclical component adds 10 units, and random fluctuations reduce the value by 5 units. The observed value becomes:

$$Y_t = 100 + 20 + 10 - 5 = 125$$

This demonstrates how different components combine to produce the final observed value.
Time series models are widely used in finance for forecasting stock prices. A simple predictive model can be expressed as:

$$Y_{t+1} = Y_t + \Delta$$

If the current stock price is 100 and an increase of 5 is expected, then:

$$Y_{t+1} = 105$$

Thus, the predicted price for the next time period is 105. Similarly, in computer science, time series models are used for system monitoring. For instance, if CPU usage increases by 10% per hour and the current usage is 50%, then:

$$CPU_{t+1} = 50 + 10 = 60\%$$

This helps system administrators anticipate resource requirements and avoid system failures.

The basic Objectives of Time Series Models are as follows:

1) **Description and understanding of data:** The first objective of time series analysis is to understand the structure and behaviour of the data. This involves identifying key components such as trend, seasonality, and randomness.
   The decomposition of a time series can be written as:

   $$Y_t = T_t + S_t + R_t$$

   where each component contributes to the overall observed value. For example, if the trend is 200 units, seasonal variation adds 30 units, and random noise subtracts 10 units, then:

   $$Y_t = 200 + 30 - 10 = 220$$

   This helps in understanding how different factors influence the data.

   Another important aspect is pattern recognition, which involves identifying trends, cycles, and anomalies. For instance, if a dataset contains values such as 100, 102, 105, and suddenly jumps to 150, the value 150 can be identified as an anomaly. Recognizing such unusual patterns is critical for improving model accuracy and detecting potential issues.

2) **Forecasting and Prediction**
   Forecasting is one of the primary objectives of time series analysis, where past observations are used to estimate future values. Forecasting can be broadly classified into short-term and long-term prediction.
   Short-term forecasting often uses simple techniques such as moving averages. The formula for a three-period moving average is:

   $$\hat{Y}_{t+1} = \frac{Y_t + Y_{t-1} + Y_{t-2}}{3}$$

   For example, if the last three observations are 100, 110, and 120, then the predicted next value is:

$$\hat{Y}_{t+1} = \frac{100 + 110 + 120}{3} = 110$$

This provides a simple yet effective estimate of the next value.

Long-term forecasting typically relies on trend models. A common form is the linear trend equation:

$$Y_t = a + bt$$

where $a$is the intercept and $b$is the slope representing the rate of change over time. For instance, if $a = 50$, $b = 10$, and $t = 5$, then:

$$Y_5 = 50 + 10(5) = 100$$

This shows how future values can be estimated based on long-term trends.

3) Control and Optimization

The final objective of time series analysis is to use predictions for control and optimization. This means applying forecasting results to improve system performance and decision-making. In predictive maintenance, time series models help estimate the likelihood of system failure. A simple model can be expressed as:

$$Risk = Base + Rate \times Time$$

For example, if the base risk is 10 and the risk increases by 2 units per time period, then after 5-time units:

$$Risk = 10 + 2(5) = 20$$

This allows maintenance to be scheduled before failure occurs.

In inventory management, forecasting demand is essential to maintain optimal stock levels. A simple demand model is:

$$Demand = Average\ + Seasonal\ Factor$$

If the average demand is 100 units and seasonal demand adds 20 units, then:

$$Demand = 120$$

This helps businesses maintain adequate inventory without overstocking.

In quality control, time series models monitor deviations from expected performance. The deviation can be calculated as:

$$Deviation = Actual - Expected$$

For example, if the expected output is 100 units and the actual output is 105 units, then:

$$Deviation = 5$$

This indicates a variation that may require corrective action.

Now we are going to discuss the following Time Series Models:

- ARIMA,
- SARIMA,
- LSTM,
- Bi-LSTM

1. ARIMA (Autoregressive Integrated Moving Average) is one of the most widely used statistical models for time series forecasting. It combines three key components: autoregression (AR), integration (I), and moving average (MA). The autoregressive part captures the relationship between the current value and its past values, the integrated part ensures the series becomes stationary through differencing, and the moving average component models the relationship between the current value and past forecast errors. This makes ARIMA particularly effective for modeling linear relationships in time-dependent data.

The general form of the ARIMA model is expressed as:

$$ARIMA(p,d,q): \phi(B)(1-B)^d Y_t = \theta(B)\varepsilon_t$$

Here, p represents the number of lag observations, d represents the degree of differencing, and q represents the size of the moving average window.

In a simplified form, it can be written as:

$$Y_t = c + \sum \phi_i Y_{t-i} + \sum \theta_j \varepsilon_{t-j} + \varepsilon_t$$

where terms have their usual meaning, we will discuss them in detail in Unit 9

ARIMA should be used when the data is stationary or can be made stationary through differencing, and when there is no strong seasonal pattern present. For example, in stock market analysis, if the current value depends on the previous value, a simple ARIMA model may be:

$$Y_t = 0.6Y_{t-1} + \varepsilon_t$$

If the previous value is 100, the predicted value becomes:

$$Y_t = 60 + \varepsilon_t$$

This type of model is commonly used in stock price forecasting and demand prediction.

2. SARIMA (Seasonal ARIMA) is an extension of ARIMA that incorporates seasonal components into the model. While ARIMA handles non-seasonal data, SARIMA is specifically designed to capture patterns that repeat over fixed intervals, such as daily, monthly, or yearly cycles. This makes it highly effective for datasets with clear seasonal trends.

The general form of SARIMA is:

$$SARIMA(p,d,q)(P,D,Q)_m$$

where (Q)represent the seasonal components and m denotes the seasonal period. The expanded form is:

$$\Phi(B^m)\phi(B)(1-B)^d(1-B^m)^D Y_t = \Theta(B^m)\theta(B)\varepsilon_t$$

where terms have their usual meaning, we will discuss them in detail in Unit 9

SARIMA is most appropriate when the data exhibits strong seasonal behaviour. For instance, retail sales often increase during festive seasons like December. If the seasonal period is 12 (monthly data), a simple interpretation is:

$$Y_t = Y_{t-12} + trend + error$$

This means that current sales are influenced by the sales from the same month in the previous year. SARIMA is widely used in applications such as sales forecasting, energy demand prediction, and tourism analysis.

3. LSTM (Long Short-Term Memory) is a type of deep learning model belonging to the family of Recurrent Neural Networks (RNNs). It is specifically designed to handle sequential data and capture long-term dependencies, overcoming the limitations of traditional RNNs such as the vanishing gradient problem. LSTM achieves this through a memory cell and three gates: the forget gate, input gate, and output gate, which regulate the flow of information.

The functioning of LSTM can be described through the following equations:

Forget gate:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Cell state update:

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t$$

Output:

$$h_t = o_t tanh\,(C_t)$$

where terms have their usual meaning, we will discuss them in detail in Unit 9

LSTM should be used when the data is non-linear, complex, and involves long-term dependencies. It is particularly effective when large datasets are available. For example, in stock price prediction, LSTM can learn patterns from historical sequences such as [100, 105, 110] and use them to predict future values. It is also widely used in natural language processing, speech recognition, and weather forecasting.

4. Bi-LSTM (Bidirectional LSTM) is an advanced extension of LSTM that processes data in both forward and backward directions. This allows the model to capture information not only from past inputs but also from future context, leading to improved performance in sequence modelling tasks. It consists of two LSTM layers: one that processes the sequence from left to right and another from right to left.

The conceptual representation of Bi-LSTM is:

Forward pass:

$$\overrightarrow{h_t} = LSTM(x_t)$$

Backward pass:

$$h_t^{\leftarrow} = LSTM(x_t)$$

Final output:

$$h_t = [\overrightarrow{h_t}, h_t^{\leftarrow}]$$

where terms have their usual meaning, we will discuss them in detail in Unit 9

Bi-LSTM should be used when understanding the context from both past and future is important. This is especially useful in applications such as natural language processing, where the meaning of a word depends on the surrounding words. For example, the word “bank” can refer to a financial institution or a riverbank, depending on the context before and after it. By analyzing the sequence in both directions, Bi-LSTM provides better contextual understanding. It is commonly used in sentiment analysis, speech recognition, and advanced forecasting tasks.

In short ARIMA and SARIMA are statistical models best suited for linear time series data, with SARIMA specifically handling seasonal patterns. On the other hand, LSTM and Bi-LSTM are deep learning models capable of capturing complex, non-linear relationships and long-term dependencies in sequential data. The choice of model depends on the nature of the dataset, the presence of seasonality, and the complexity of the patterns to be learned.

Comparison Between ARIMA & SARIMA

ARIMA (Autoregressive Integrated Moving Average) and SARIMA (Seasonal ARIMA) are both widely used time series forecasting models, but they differ primarily in their ability to handle seasonality and model complexity.

The comparison is as follows:

- First, ARIMA is designed to model **non-seasonal time series data**, whereas SARIMA is an extension of ARIMA that can handle **seasonal patterns** in addition to non-seasonal components. In other words, SARIMA builds upon ARIMA by incorporating seasonal behavior present in the data.

- Another key difference lies in seasonality handling. ARIMA does not explicitly account for seasonal variations, meaning it is suitable only when the data does not exhibit repeating seasonal patterns. In contrast, SARIMA is specifically designed to capture such patterns by incorporating seasonal differencing and seasonal lag terms.

- From a complexity perspective, ARIMA is simpler and computationally less expensive because it involves fewer parameters. SARIMA, however, is more complex due to the addition of seasonal components, which increases both computational cost and model tuning effort.

- Regarding data requirements, both models require the data to be stationary. However, SARIMA may require **additional seasonal differencing** to remove seasonal trends, whereas ARIMA only focuses on non-seasonal stationarity.

- In terms of use cases, ARIMA is suitable for datasets where there is **no clear seasonal behavior**, such as short-term stock returns or non-cyclical demand forecasting. SARIMA, on the other hand, is more appropriate when the data exhibits **regular repeating patterns**, such as monthly sales data, electricity consumption, or tourism demand that follows yearly cycles.

- Finally, in terms of interpretability, ARIMA is generally easier to understand due to its simpler structure, while SARIMA can be slightly more complex because of the additional seasonal parameters.

Overall, ARIMA is best suited for **non-seasonal, linear time series data**, while SARIMA should be used when the data contains **clear seasonal patterns**. Essentially, SARIMA can be viewed as an enhanced version of ARIMA that adds the ability to model seasonality, making it more powerful for real-world time series data that exhibit periodic behavior.

Comparison between LSTM & Bi LSTM

LSTM (Long Short-Term Memory) and Bi-LSTM (Bidirectional LSTM) are both advanced variants of Recurrent Neural Networks designed to handle sequential data and capture dependencies over time. However, they differ primarily in the way they process input sequences and utilize contextual information.

The comparison is as follows:

- First, LSTM processes data in a **single direction**, typically from past to future. This means that at any given time step, the prediction is based only on the current input and the previous hidden states. In contrast, Bi-LSTM processes the sequence in **both forward and backward directions**, allowing it to capture information from both past and future contexts simultaneously.

- In terms of structure, a standard LSTM consists of one sequence of memory cells connected through time, controlled by gates such as the forget gate, input gate, and output gate. Bi-LSTM, however, consists of **two LSTM layers**—one that processes the sequence forward and another that processes it backward—and their outputs are combined to produce the final result.

- From a performance perspective, LSTM is effective in capturing **long-term dependencies**, but it may miss important context that lies ahead in the sequence. Bi-LSTM overcomes this limitation by considering both past and future information, often leading to **better accuracy and richer feature representation**, especially in tasks where context is critical.

- In terms of computational complexity, LSTM is relatively less complex and faster to train because it processes the sequence only once. Bi-LSTM, on the other hand, is **more computationally expensive** as it requires two passes through the data and combines the results, making it slower and more resource-intensive.

- Regarding use cases, LSTM is suitable when predictions depend mainly on **past information**, such as stock price prediction, time series forecasting, or sensor data analysis. Bi-LSTM is more appropriate when understanding the **full context of the sequence (both past and future)** is important, such as in natural language processing tasks like sentiment analysis, speech recognition, and text classification.

- Finally, in terms of applicability, LSTM is often preferred for **real-time or streaming data**, where future data is not available. Bi-LSTM, however, is better suited for **offline analysis**, where the entire sequence is available beforehand and both directions can be utilized.

Overall, LSTM is a unidirectional model that captures past dependencies and is efficient for real-time prediction tasks, whereas Bi-LSTM is a bidirectional model that leverages both past and future context to achieve higher accuracy in complex sequence modelling tasks.

Finally, in this section we learned that the Time series analysis is a powerful approach for understanding and forecasting data that evolves over time. By decomposing data into components such as trend and seasonality, identifying patterns, and applying forecasting techniques, it becomes possible to make informed predictions. These predictions are then used for optimization in areas such as finance, system monitoring, inventory management, and industrial processes. Understanding both the mathematical

foundation and practical application of time series models is essential for effective data-driven decision-making.

☞ **Check Your Progress 2**

Q1. What is Time Series Analysis? How does it differ from general statistical analysis?

.....

.....

Q2. Explain the components of a Time Series.

.....

.....

Q3. What are the main objectives of Time Series Analysis?

.....

.....

Q4. Differentiate between ARIMA and SARIMA models.

.....

.....

Q5. Compare LSTM and Bi-LSTM models in time series forecasting.

.....

.....

7.4 SUPERVISED LEARNING MODELS - AN INTRODUCTION

Supervised learning is a fundamental concept in machine learning where models are trained using labelled data, meaning that each input is associated with a known output. The primary objective of supervised learning is to learn a mapping function that can accurately predict the output for new, unseen inputs. This relationship between input and output can be mathematically represented as:

$$y = f(x)$$

where $x$represents the input features and $y$represents the target variable. The model learns this function during training and uses it for prediction.

An important aspect of supervised learning is its ability to generalize beyond the training data. This is measured using the concept of generalization error, which reflects how well the model performs on unseen data. For instance, consider a simple model used to predict house prices:

$$y = 50000 + 1000x$$

If the size of a house is $x = 100$, then the predicted price becomes:

$$y = 50000 + 1000(100) = 150000$$

This example shows how supervised learning models can learn patterns from data and use them to make predictions.

In practical applications, supervised learning is widely used in domains such as finance, healthcare, and computer vision. The process typically involves splitting the dataset into training and testing sets, where the training set is used to build the model and the testing set is used to evaluate its performance.

Supervised learning tasks are broadly categorized into classification and regression, depending on the nature of the output variable.

Classification Tasks i.e. Predicting Discrete Outcomes, involve tasks for predicting categorical or discrete outcomes, where the output belongs to a predefined set of classes. The objective is to assign the correct label to each input based on learned patterns.
Mathematically, classification models often estimate the probability of a class given the input features:

$$P(y \mid x)$$

The predicted class is typically the one with the highest probability.
For example, consider an email classification system that predicts whether an email is spam or not. If the model outputs:

$$P(Spam \mid x) = 0.8$$

Then, since the probability is greater than 0.5, the email is classified as Spam. Another simple rule-based example can be expressed as:

$$y = \{1 \; if \; x > 50 \; 0 \; otherwise$$

If $x = 60$, then the output is $y = 1$, indicating the email is spam. This demonstrates how classification models assign discrete labels.

Regression Tasks i.e. Predicting Continuous Values are the tasks that focus on predicting continuous numerical values such as prices, temperature, or sales. The goal is to minimize the difference between actual and predicted values using a loss function.
One of the most commonly used loss functions is the Mean Squared Error (MSE), defined as:

$$MSE = \frac{1}{n}\sum(y_i - \hat{y}_i)^2$$

This measures the average squared difference between actual values and predicted values.
For example, suppose the actual values are [100, 120] and the predicted values are [110, 115]. Then the MSE is calculated as:

$$MSE = \frac{(100 - 110)^2 + (120 - 115)^2}{2} = \frac{100 + 25}{2} = 62.5$$

This value indicates the prediction error, and lower values represent better model performance.

The Supervised Learning Workflow follows an iterative workflow that includes selecting an appropriate algorithm, training the model, evaluating its performance, and refining it. The goal is to achieve a balance between model accuracy, computational efficiency, and interpretability.

A key concept in this process is the bias-variance trade-off. Bias refers to errors due to overly simplistic models, while variance refers to errors due to overly complex models that fit the training data too closely.
For example, if the actual value is 100 and one model predicts 80, it indicates high bias (underfitting). If another model predicts 98, it indicates lower bias and better performance. Thus, selecting the right level of model complexity is essential.

Supervised model training and learning rely on mathematical foundations that guide how models learn from data. The first important concept is the loss function, which measures the error between actual and predicted values. A common loss function is:

$$L(y, \hat{y}) = (y - \hat{y})^2$$

For example, if the actual value is 50 and the predicted value is 45, then:

$$L = (50 - 45)^2 = 25$$

This error guides the model in improving its predictions.

Another important concept is gradient descent, an optimization technique used to minimize the loss function. The update rule is:

$$\theta = \theta - \alpha \frac{\partial L}{\partial \theta}$$

For example, if $\theta = 5$, learning rate $\alpha = 0.1$, and gradient = 2, then:

$$\theta = 5 - 0.1(2) = 4.8$$

This shows how model parameters are updated iteratively to reduce error.

Finally, the bias-variance trade-off plays a crucial role in determining model performance. The total error can be expressed as:

$$Error = Bias^2 + Variance + Noise$$

A model with high bias may underfit the data, while a model with high variance may overfit. Therefore, achieving a balance between bias and variance is essential for building robust models.

Supervised learning is a powerful framework for predictive modeling that relies on labeled data to learn relationships between inputs and outputs. It encompasses classification and regression tasks, supported by mathematical foundations such as loss functions and optimization techniques. By understanding these concepts and carefully managing trade-offs like bias and variance, supervised learning enables accurate and reliable predictions across a wide range of real-world applications.

In General, the Supervised Learning Models include following:

1) K-Nearest neighbour
2) naïve-bayes
3) Logistic regression
4) decision trees
5) Random Forest
6) Support vector Machines

Now, we will discuss these Supervised Learning Models in brief

1. K-Nearest Neighbours (KNN)

K-Nearest Neighbours (KNN) is a simple and intuitive supervised learning algorithm that belongs to the category of non-parametric and instance-based learning methods. Instead of building a mathematical model during training, KNN stores all the training data and makes predictions based on the similarity

between data points. When a new data point is introduced, the algorithm identifies the $k$nearest neighbors and assigns the class based on majority voting.

The similarity between data points is usually measured using distance metrics such as Euclidean distance, which is given by:

$$d = \sqrt{\sum(x_i - y_i)^2}$$

KNN is best suited for small datasets where there are no strong assumptions about the underlying data distribution, and where patterns are determined by proximity between points. For example, in customer segmentation, if a new customer is surrounded by three nearest customers who are classified as "high spenders," then the new customer is also classified as a high spender. This demonstrates how KNN relies purely on neighbourhood information.

2. Naïve Bayes

Naïve Bayes is a probabilistic classification algorithm based on Bayes' theorem. It assumes that all features are independent of each other given the class label, which simplifies the computation significantly. Despite this "naïve" assumption, the model performs very well in many real-world scenarios, especially in text classification tasks.

The core formula of Naïve Bayes is:

$$P(C \mid X) = \frac{P(X \mid C)P(C)}{P(X)}$$

where $P(C \mid X)$is the posterior probability, $P(X \mid C)$is the likelihood, and $P(C)$is the prior probability. Naïve Bayes is particularly effective when working with large datasets and independent features, such as in spam detection or sentiment analysis. For instance, if the probability that an email is spam given certain words is calculated as:

$$P(Spam \mid words) = 0.9$$

then the email is classified as spam. This shows how probability-based reasoning is used for classification.

3. Logistic Regression

Logistic regression is a supervised learning algorithm used primarily for binary classification problems. Unlike linear regression, which predicts continuous values, logistic regression predicts probabilities using a sigmoid (logistic) function, ensuring that the output lies between 0 and 1.

The mathematical form of logistic regression is:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$$

This probability is then converted into a class label based on a threshold (usually 0.5).

Logistic regression is suitable when the data is linearly separable and when probability interpretation is important. For example, in loan approval systems, if the predicted probability of approval is 0.7, which is greater than 0.5, the loan is approved. This makes logistic regression highly interpretable and widely used in finance and healthcare.

4. Decision Trees

Decision trees are powerful supervised learning models that use a tree-like structure to make decisions. The dataset is split into subsets based on feature values, forming branches and nodes until a final decision (leaf node) is reached. This makes decision trees highly interpretable and easy to visualize.

A common criterion used for splitting is entropy, defined as:

$$Entropy = -\sum p_i log_{\,2}\, p_i$$

Decision trees are suitable for handling both linear and non-linear relationships and can be used for classification as well as regression tasks. They are especially useful when interpretability is important. For example, in a loan approval system, a decision tree may follow rules such as: if income is greater than 50,000, approve the loan; otherwise, reject it. This rule-based approach makes the decision-making process transparent.

5. Random Forest

Random Forest is an ensemble learning technique that improves upon decision trees by combining multiple trees to produce a more accurate and stable prediction. Each tree is built using a random subset of the data and features, and the final prediction is obtained by aggregating the outputs of all trees.
The conceptual formula for Random Forest prediction is:

$$Final\ Prediction = \sum\ \ Predictions\ of\ Tree$$

Random Forest is particularly useful when dealing with large datasets and when individual decision trees tend to overfit. By averaging multiple trees, it reduces variance and improves generalization. For example, in credit scoring, multiple decision trees may evaluate different aspects of a customer's profile, and the final decision is based on the majority vote or average prediction.

6. Support Vector Machines (SVM)

Support Vector Machines (SVM) is a powerful supervised learning algorithm that aims to find the optimal hyperplane that separates different classes with the maximum margin. The idea is to maximize the distance between the nearest data points (support vectors) and the decision boundary.
The equation of the hyperplane is:

$$w \cdot x + b = 0$$

and the objective is to maximize the margin:

$$\frac{2}{||\, w \,||}$$

SVM is particularly effective in high-dimensional spaces and can handle both linear and non-linear data using kernel functions. It is best used when there is a clear separation between classes and when the dataset is not extremely large. For example, in face recognition systems, SVM can separate different faces by constructing an optimal boundary in high-dimensional feature space.

Finally, we learned that the Supervised learning models provide a wide range of techniques for solving classification and regression problems. KNN relies on proximity, Naïve Bayes uses probability, logistic regression predicts probabilities for classification, decision trees use rule-based splitting, Random Forest enhances accuracy through ensemble learning, and SVM focuses on maximizing the margin between classes. The choice of model depends on the nature of the dataset, the complexity of relationships, and the desired balance between accuracy and interpretability.

This section concludes with the understanding that Supervised learning is a powerful framework for predictive modelling that relies on labelled data to learn relationships between inputs and outputs. It encompasses classification and regression tasks, supported by mathematical foundations such as loss functions and optimization techniques. By understanding these concepts and carefully managing trade-offs like bias and variance, supervised learning enables accurate and reliable predictions across a wide range of real-world applications.

The next section we need to address is Unsupervised learning, before starting this section on Unsupervised learning we need to understand the differences between Supervised learning and Unsupervised learning.

Delineating Unsupervised from Supervised Learning

The primary distinction between supervised and unsupervised learning lies in the nature of the data. Supervised learning uses labelled data, while unsupervised learning works with unlabelled data.

The key difference between supervised and unsupervised learning lies in the availability of labeled outputs. Supervised learning uses labeled data to learn a mapping function:

$$y = f(x)$$

where both input and output are known during training. In contrast, unsupervised learning does not involve a target variable and instead focuses on discovering patterns within the data.

For example, in supervised learning, an input such as house size may be mapped to a known price. In unsupervised learning, similar data points may be grouped into clusters without predefined labels.

In contrast, unsupervised learning focuses on discovering hidden structures:

$$Find\ f(X)$$

For example, consider the dataset [1, 2, 50, 55]. Without labels, an unsupervised algorithm groups similar values together:

- Cluster 1 → [1, 2]
- Cluster 2 → [50, 55]

This grouping is based purely on similarity, demonstrating how unsupervised learning identifies natural patterns in data.

Now, let's understand the Unsupervised learning models in the next section.

☞ Check Your Progress 3

Q1. What is Supervised Learning? Explain its objective in predictive analytics.

……

……

Q2. Differentiate between Classification and Regression tasks in supervised learning.

……

……

Q3. Explain the concept of loss function and its role in supervised learning.

……

……

Q4. What is the bias-variance trade-off? Why is it important?.

……

……

Q5. List and briefly explain any two supervised learning algorithms.

.....

.....

7.5 UNSUPERVISED LEARNING MODELS - AN INTRODUCTION

Unsupervised learning is a fundamental paradigm in machine learning that focuses on extracting meaningful insights from data that does not contain predefined labels. Unlike supervised learning, where models are trained using input-output pairs, unsupervised learning works solely with input data and aims to uncover hidden patterns, structures, or relationships within it.
Mathematically, unsupervised learning can be represented as:

$$X = \{x_1, x_2, \ldots, x_n\}$$

where only the input dataset X is available, and the objective is to discover an underlying structure $f(X)$. This makes unsupervised learning particularly suitable for exploratory data analysis, where the goal is to understand the data rather than predict known outcomes.

The primary tasks in unsupervised learning include clustering, association rule mining, and dimensionality reduction. Clustering groups have similar data points, association rules identify relationships between variables, and dimensionality reduction simplifies complex datasets while preserving essential information. i.e. The primary objective of unsupervised learning is **exploratory data analysis**, where algorithms identify similarities, group data points, and reveal hidden insights. These methods are broadly categorized into three major tasks:

- **Clustering** → grouping similar data points
- **Association Rules** → finding relationships between variables
- **Dimensionality Reduction** → simplifying data representation

The core of Unsupervised learning starts by Defining the Unlabelled Data Paradigm, in the unlabelled data paradigm, algorithms rely entirely on the inherent properties of the data to identify meaningful groupings or structures. Since no prior labels are available, the model must determine similarity based on distance or statistical measures.

A common objective in clustering is to minimize the distance between data points and their respective cluster centroids, expressed as:

$$Minimize: \sum_{i=1}^{k} \sum_{x \in C_i} || x - \mu_i ||^2$$

where C_i represents a cluster and μ_i represents its centroid.

For example, consider data points [2, 4, 10] divided into two clusters. The centroid of the first cluster containing points 2 and 4 is:

$$\mu_1 = \frac{2+4}{2} = 3$$

The distance of each point from the centroid is:

$$|| 2 - 3 ||= 1, || 4 - 3 ||= 1$$

The point 10 forms a separate cluster with centroid 10. This illustrates how clustering organizes data based on similarity without requiring labels.

It is to be noted that Unsupervised learning plays a significant role in extracting actionable insights from raw data, making it highly valuable in real-world applications. We can understand better by considering some Real world application of unsupervised learning techniques to perform exploratory data analysis and generate some insights.

Say, for example, in customer segmentation, clustering algorithms group customers based on purchasing behaviour or demographics. For instance, given spending values [100, 120, 500], the algorithm may group [100, 120] as low spenders and [500] as high spenders. The similarity between customers is often measured using distance metrics such as:

$$d = \sqrt{(x_2)^2}$$

Similarly, in recommendation systems, association rules are used to discover relationships between items. These relationships are quantified using metrics such as support, confidence, and lift.

$$Support = \frac{Transactions\ containing\ A}{Total\ Transactions}$$

$$Confidence = \frac{Support(A \cap B)}{Support(A)}$$

$$Lift = \frac{Confidence}{Support(B)}$$

For example, if 20 out of 100 transactions contain item A, then:

$$Support = \frac{20}{100} = 0.2$$

If 15 of these transactions also contain item B, then:

$$Confidence = \frac{15}{20} = 0.75$$

A high confidence value indicates a strong association between items.

Also, Unsupervised learning is also widely used in anomaly detection to identify unusual data points.

This can be done using the Z-score formula:

$$Z = \frac{x - \mu}{\sigma}$$

For example, if the mean is 50, the standard deviation is 10, and a value of 100 is observed, then:

$$Z = \frac{100 - 50}{10} = 5$$

Since this value is far from the mean, it is identified as an outlier.

We learned about the basics of unsupervised learning, but it is to be noted that the **theoretical objective of unsupervised learning** is to understand the mathematical and algorithmic foundations behind pattern discovery techniques.

The most frequently used techniques involve the K-Means clustering, Hierarchical Clustering, Association rule mining technique.

In K-Means clustering, the centroid of a cluster is calculated as:

$$\mu_k = \frac{1}{n_k}\sum x_i$$

For example, if the data points are [2, 4], the centroid becomes:

$$\mu = \frac{2+4}{2} = 3$$

But, Hierarchical clustering builds clusters step by step based on distance. For instance, given points 2, 3, and 10, the closest points (2 and 3) are merged first.

Whereas, the Apriori algorithm is used in association rule mining to identify frequent itemsets. The support of an item is calculated as:

$$Support(A) = \frac{Count(A)}{Total\ Transactions}$$

For example, if an item appears 30 times in 100 transactions:

$$Support = 0.3$$

Further**, the Analytical Objectives of Unsupervised learning** involves the Evaluation and Interpretation of the results i.e. the Analytical objectives focus on evaluating the quality of discovered patterns and clusters.

One commonly used metric is the silhouette score, defined as:

$$S = \frac{b-a}{max\ (a,b)}$$

where a is the average intra-cluster distance and b is the nearest cluster distance.
For example, if $a = 2$ and $b = 5$, then:

$$S = \frac{5-2}{5} = 0.6$$

This indicates good clustering performance.

Similarly, association rules are evaluated using support, confidence, and lift, where a lift value greater than 1 indicates a strong and meaningful relationship between items.

However**, the Practical Objectives of Unsupervised learning** relates to Model Selection and their limitations i.e. the practical objective of unsupervised learning is to select appropriate models based on data characteristics and understand their limitations.

For example, the K-Means clustering is sensitive to outliers i.e. If the dataset is [1, 2, 100], the centroid shifts significantly:

$$\mu = \frac{1 + 2 + 100}{3} \approx 34$$

This demonstrates how outliers can distort results.

Alternative models such as density-based clustering (e.g., DBSCAN) address this limitation by grouping points based on density. In this approach, points are assigned to a cluster if their distance is less than a threshold ϵ. Points that do not satisfy this condition are treated as noise.

So, we learned that Unsupervised learning provides a powerful framework for analysing unlabelled data and discovering hidden structures within it. By leveraging techniques such as clustering, association rule mining, and anomaly detection, it enables meaningful insights without the need for predefined outputs. The use of mathematical formulations, evaluation metrics, and real-world applications makes unsupervised learning an essential tool in modern data analysis, particularly in scenarios where labelled data is unavailable or costly to obtain.

Now we are going to discuss in brief, about the Unsupervised Learning Models:

1. K-Means Clustering
2. Hierarchical Clustering
3. Agglomerative Clustering
4. Association Rules

1. K-Means Clustering is one of the most widely used unsupervised learning algorithms, based on the concept of partitioning data into a predefined number of clusters. It is a centroid-based method in which each cluster is represented by its centroid, and the algorithm aims to minimize the distance between data points and their respective centroids. The process is iterative: initially, centroids are selected, then data points are assigned to the nearest centroid, and finally, centroids are recalculated until the clusters stabilize.
The objective function of K-Means is to minimize the within-cluster sum of squares, expressed as:

$$Minimize: \sum_{i=1}^{k} \sum_{x \in C_i} || x - \mu_i ||^2$$

where $C_i$represents the cluster and $\mu_i$represents its centroid. The centroid itself is calculated as:

$$\mu_i = \frac{1}{n} \sum x_i$$

K-Means is most suitable when the data is numerical, well-separated, and when the number of clusters $k$is known in advance. It performs efficiently on large datasets but may struggle with irregular cluster shapes or outliers. For example, in customer segmentation, if the spending values are [100, 120, 500], the algorithm may group [100, 120] as low spenders and [500] as high spenders, illustrating how clustering is based on similarity.

2. Hierarchical Clustering is a method that builds a hierarchy of clusters, either by merging smaller clusters into larger ones or by splitting larger clusters into smaller ones. Unlike K-Means, it does not require the number of clusters to be specified in advance. The result is typically represented using a dendrogram, which visually shows the arrangement of clusters and their relationships.
The similarity between data points is commonly measured using distance metrics such as Euclidean distance:

$$d = \sqrt{\sum(x_i - y_i)^2}$$

Hierarchical clustering is particularly useful for small datasets and when there is a need to understand the relationships between clusters at different levels. It is also beneficial when the number of clusters is unknown. For instance, given data points [2, 3, 10], the algorithm identifies that 2 and 3 are closest and merges them first, forming a cluster before considering other points. This step-by-step merging process builds a hierarchy of clusters.

3. Agglomerative Clustering is a specific type of hierarchical clustering that follows a bottom-up approach. In this method, each data point initially forms its own cluster, and pairs of clusters are merged iteratively based on similarity until all points belong to a single cluster or a stopping condition is met. One commonly used method to measure the distance between clusters is single linkage, defined as:

$$d(A, B) = min\ (d(x, y))$$

where $x$and $y$are points in clusters $A$and B, respectively.
Agglomerative clustering is suitable when interpretability and hierarchical structure are important, and it works well for small to medium-sized datasets. It does not assume any specific shape of clusters, making it flexible in various scenarios. For example, in document clustering, different articles are initially treated as separate clusters and gradually grouped together based on similarity in content, forming meaningful clusters over time.

4. Association Rules learning is an unsupervised technique used to discover interesting relationships or patterns between variables in large datasets. It is widely used in market basket analysis, where the goal is to identify items that are frequently purchased together.
The strength of association rules is measured using metrics such as support, confidence, and lift. Support indicates how frequently an item appears in the dataset:

$$Support(A) = \frac{Transactions\ containing\ A}{Total\ Transactions}$$

Confidence measures the likelihood that item B is purchased when item A is purchased:

$$Confidence(A \rightarrow B) = \frac{Support(A \cap B)}{Support(A)}$$

Lift evaluates the strength of the association relative to random occurrence:

$$Lift = \frac{Confidence}{Support(B)}$$

Association rules are most useful in large transactional datasets, recommendation systems, and retail analysis. For example, if there are 100 transactions, and 20 include bread while 15 include both bread and butter, then:

$$Confidence = \frac{15}{20} = 0.75$$

This indicates a strong association between bread and butter, suggesting that customers who buy bread are likely to also buy butter.

This section concludes with the understanding that the Unsupervised learning models such as K-Means, hierarchical clustering, agglomerative clustering, and association rules play a crucial role in discovering hidden patterns in unlabelled data. K-Means provides efficient clustering based on centroids, hierarchical methods reveal relationships between clusters, agglomerative clustering builds clusters step

by step, and association rules uncover meaningful relationships between variables. The choice of method depends on the dataset size, structure, and the type of insights required.

☞ Check Your Progress 4

Q1. What is Unsupervised Learning? Explain its objective in predictive analytics.

.....

.....

Q2. Differentiate between Supervised and Unsupervised Learning.

.....

.....

Q3. Explain Clustering and its importance in unsupervised learning.

.....

.....

Q4. What are the common unsupervised learning techniques? Briefly explain any two.

.....

.....

Q5. What are the challenges in Unsupervised Learning?

.....

.....

7.6 SUMMARY

Unit 7 on Predictive Analytics Models provides a comprehensive introduction to the techniques used for forecasting future outcomes based on historical data. The unit begins with regression models, which establish relationships between variables and are widely used for predicting continuous values such as sales, demand, or risk. These models form the foundation of predictive analytics by quantifying how changes in independent variables influence a dependent variable, enabling informed forecasting and decision-making.

The unit further explores time series and forecasting models, which focus on analysing data collected over time to identify trends, seasonality, and patterns. These models are essential for predicting future values in time-dependent scenarios such as stock prices, weather patterns, and business performance. By understanding temporal structures in data, learners can develop accurate and reliable forecasts that support planning and strategic decisions.

In addition, the unit introduces supervised and unsupervised learning models, which are key components of modern machine learning. Supervised learning uses labelled data to train models for prediction and classification tasks, while unsupervised learning identifies hidden patterns, groupings, or structures in unlabelled data. Together, these approaches provide a complete framework for predictive analytics, enabling learners to apply both statistical and machine learning techniques to real-world problems and make data-driven predictions effectively.

7.7 ANSWERS

☞ Check Your Progress 1

Q1. What is Regression Analysis? Explain its objective in predictive analytics.
Solution: Refer to Section 7.2
Q2. Differentiate between dependent and independent variables with examples.
Solution: Refer to Section 7.2
Q3. Explain Linear Regression with its mathematical form and example.
Solution: Refer to Section 7.2
Q4. What are the different types of regression models? Briefly explain any two.
Solution: Refer to Section 7.2
Q5. Discuss the challenges in regression analysis.
Solution: Refer to Section 7.2

☞ Check Your Progress 2

Q1. What is Time Series Analysis? How does it differ from general statistical analysis?
Solution: Refer to Section 7.3
Q2. Explain the components of a Time Series.
Solution: Refer to Section 7.3
Q3. What are the main objectives of Time Series Analysis?
Solution: Refer to Section 7.3
Q4. Differentiate between ARIMA and SARIMA models.
Solution: Refer to Section 7.3
Q5. Compare LSTM and Bi-LSTM models in time series forecasting.
Solution: Refer to Section 7.3

☞ Check Your Progress 3

Q1. What is Supervised Learning? Explain its objective in predictive analytics.
Solution: Refer to Section 7.4
Q2. Differentiate between Classification and Regression tasks in supervised learning.
Solution: Refer to Section 7.4
Q3. Explain the concept of loss function and its role in supervised learning.
Solution: Refer to Section 7.4
Q4. What is the bias-variance trade-off? Why is it important?.
Solution: Refer to Section 7.4
Q5. List and briefly explain any two supervised learning algorithms.
Solution: Refer to Section 7.4

☞ Check Your Progress 4

Q1. What is Unsupervised Learning? Explain its objective in predictive analytics.
Solution: Refer to Section 7.5
Q2. Differentiate between Supervised and Unsupervised Learning.
Solution: Refer to Section 7.5
Q3. Explain Clustering and its importance in unsupervised learning.
Solution: Refer to Section 7.5
Q4. What are the common unsupervised learning techniques? Briefly explain any two.
Solution: Refer to Section 7.5
Q5. What are the challenges in Unsupervised Learning?
Solution: Refer to Section 7.5

7.8 REFERENCES/FURTHER READINGS

1. *Pattern Recognition and Machine Learning* by Christopher M. Bishop, published by Springer (1st Edition, 2006, ISBN: 978-0387310732).
2. *Data Mining: Concepts and Techniques* by Jiawei Han, Micheline Kamber, and Jian Pei, published by Morgan Kaufmann (3rd Edition, 2011, ISBN: 978-0123814791)
3. *An Introduction to Statistical Learning with Applications in R by Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani, published by Springer (2nd Edition, 2021, ISBN: 978-1071614174).*
4. *Practical Statistics for Data Scientists by Peter Bruce, Andrew Bruce, and Peter Gedeck, published by O'Reilly Media (2nd Edition, 2020, ISBN: 978-1492072942).*
5. *Business Analytics: Data Analysis and Decision Making* by S. Christian Albright and Wayne L. Winston, published by Cengage Learning (7th Edition, 2020, ISBN: 978-0357131787).
6. *Introduction to Operations Research* by Frederick S. Hillier and Gerald J. Lieberman, published by McGraw-Hill Education (10th Edition, 2014, ISBN: 978-0073523453).
7. *Decision Analysis for Management Judgment* by Paul Goodwin and George Wright, published by Wiley (5th Edition, 2014, ISBN: 978-1119974017).
8. *Handbook of Statistical Analysis and Data Mining Applications* by Robert Nisbet, John Elder, and Gary Miner, published by Elsevier (2nd Edition, 2017, ISBN: 978-0124166455).
9. *Data Mining and Analysis: Fundamental Concepts and Algorithms* by Mohammed J. Zaki and Wagner Meira Jr., published by Cambridge University Press (1st Edition, 2014, ISBN: 978-0521766333).
10. *Core Concepts in Data Analysis: Summarization, Correlation and Visualization* by *Boris Mirkin, published by Springer (1st Edition, 2011, ISBN: 978-0857292872).*
11. *Applied Predictive Modeling, Max Kuhn and Kjell Johnson, 1st Edition, Springer, 2013.*
12. *An Introduction to Statistical Learning with Applications in R, Gareth James, Daniela Witten, Trevor Hastie and Robert Tibshirani, 1st Edition, Springer, 2013.*
13. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction, Trevor Hastie, Robert Tibshirani and Jerome Friedman, 2nd Edition, Springer, 2009.*
14. *An Introduction to Statistical Learning with Applications in R by Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani, published by Springer (2nd Edition, 2021, ISBN: 978-1071614174).*
15. *Time Series Analysis: Forecasting and Control by George E. P. Box, Gwilym M. Jenkins, Gregory C. Reinsel, and Greta M. Ljung, published by Wiley (5th Edition, 2015, ISBN: 978-1118675021)*